1. Methods

1.1. Core Idea

1.1.1. Setup

Consider a logistic regression, with missing features in the observation.
Let $x := (x^o, x^m) \in \mathbb{R}^d$. Then the logistic regression is defined as

$$p(y \mid x) := \sigma(w^T x), \qquad w \in \mathbb{R}^d.$$

Additionally we have a PC that models the density $p(x)$.

1.1.2. Goal

Compute the prediction in a probabilistically principled manner using

$$p(y \mid x^o) := \mathbb{E}_{p(x^m \mid x^o)} \left[\sigma \left(w^T x \right) \right].$$

If it were not for the non-linear sigmoid function σ, we would be able to calculate this in a smooth and decomposable PC, by pulling the Expectation -which is linear- into the leaf nodes. To tackle this we propose to use two tricks.

1.1.3. Trick 1: Pólya-Gamma

To replace the sigmoid function we use the PG-Sigmoid identity, shown in [1], that states:

$$\sigma(z) = \frac{1}{2} \exp\left(\frac{1}{2} z\right) \; \mathbb{E}_{\gamma \sim \mathrm{PG}(1,0)} \left[\exp\left(-\frac{1}{2} \gamma z^2 \right) \right]$$

for $z \in \mathbb{R}$.

If we set $z := w^T x$, and insert into our expectation, we get:

$$\begin{aligned} p(y \mid x^o) &= \mathbb{E}_{p(x^m|x^o)}\left[\sigma(z)\right] \\ &= \mathbb{E}_{p(x^m|x^o)}\left[\frac{1}{2}\exp\left(\frac{1}{2}z\right)\mathbb{E}_{\gamma\sim\mathrm{PG}(1,0)}\left[\exp\left(-\frac{1}{2}\gamma z^2\right)\right]\right] \\ &= \frac{1}{2}\mathbb{E}_{p(x^m|x^o)}\mathbb{E}_{\gamma\sim\mathrm{PG}(1,0)}\left[\exp\left(\frac{1}{2}z - \frac{1}{2}\gamma z^2\right)\right] \end{aligned}$$

Intuitively, this means the sigmoid is replaced with a lot of bell-shaped curves averaged together, one for every value of γ. Figure 1.1 shows how well different number of bells approximate σ.

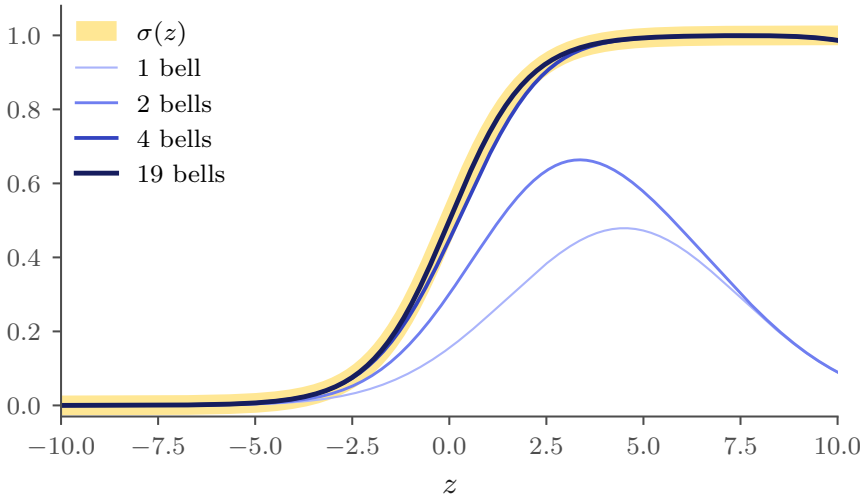


Figure 1.1.: The Pólya-Gamma identity graphically. The sigmoid represented as an average of bells $\sigma(z) = \mathbb{E}_\gamma\left[\frac{1}{2}e^{\frac{z}{2}-\frac{\gamma z^2}{2}}\right]$.

Now, we have a term without a sigmoid, but the x still appears in a quadratic form.

1.1.4. Trick 2: Hubbard-Stratonovich (HS)

To get rid of the quadratic form, we apply the Hubbard–Stratonovich identity [2, 3], that states:

$$\exp\left(-\tfrac{1}{2}ah^2\right) = \sqrt{\frac{1}{2\pi a}}\int_{-\infty}^{\infty}\exp\left(-\frac{u^2}{2a} - ihu\right)\mathrm{d}u, \tag{1.1}$$

Since $\frac{1}{\sqrt{2\pi a}} \exp\left(-\frac{u^2}{2a}\right)$ is the density of $\mathcal{N}(0, a)$, the integral is exactly the expectation of e^{-ihu} over $u \sim \mathcal{N}(0, a)$:

$$\exp\left(-\tfrac{1}{2}ah^2\right) = \mathbb{E}_{u\sim\mathcal{N}(0,\,a)}\left[e^{-ihu}\right]. \tag{1.2}$$

The trick makes a quadratic term linear at the cost of introducing a helper variable that we need to integrate over. In our term, x appears quadratically as well as linearly. To be able to apply HS, we complete the square (again using $z = w^T x$), where we write $c := \frac{1}{2\gamma}$:

$$\begin{aligned}
\frac{1}{2}z - \frac{1}{2}\gamma z^2 &= -\frac{1}{2}\gamma\left(z^2 - \frac{z}{\gamma}\right) \\
&= -\frac{1}{2}\gamma\left(z^2 - 2cz\right) \\
&= -\frac{1}{2}\gamma\left((z-c)^2 - c^2\right) \\
&= -\frac{1}{2}\gamma\,(z-c)^2 + \frac{1}{2}\gamma c^2 \\
&= -\frac{1}{2}\gamma\,(z-c)^2 + \frac{1}{8\gamma}.
\end{aligned} \tag{1.3}$$

Now we apply HS with $a = \gamma$ and $h = z - c$:

$$\begin{aligned}
\exp\left(\frac{1}{2}z - \frac{1}{2}\gamma z^2\right) &= \exp\left(\frac{1}{8\gamma}\right)\exp\left(-\frac{1}{2}\gamma\,(z-c)^2\right) \\
&= \exp\left(\frac{1}{8\gamma}\right)\mathbb{E}_{u\sim\mathcal{N}(0,\gamma)}\left[\exp\left(-iu\,(z-c)\right)\right]
\end{aligned} \tag{1.4}$$

Substituting back into $p(y \mid x^o)$, we get:

$$\begin{aligned}
p(y \mid x^o) &= \frac{1}{2}\mathbb{E}_{p(x^m|x^o)}\mathbb{E}_{\gamma\sim\mathrm{PG}(1,0)}\left[\exp\left(\frac{1}{2}z - \frac{1}{2}\gamma z^2\right)\right] \\
&= \frac{1}{2}\mathbb{E}_{p(x^m|x^o)}\mathbb{E}_{\gamma\sim\mathrm{PG}(1,0)}\left[\exp\left(\frac{1}{8\gamma}\right)\mathbb{E}_{u\sim\mathcal{N}(0,\gamma)}\left[\exp\left(-iu\,(z-c)\right)\right]\right]
\end{aligned}$$

Intuitively, this means every bell is replaced with a lot of cosine waves $\cos(u(z-c))$ averaged together, one for every value of u (see Appendix A.1 for the derivation).

Figure 1.2 shows how well different numbers of waves approximate a bell.

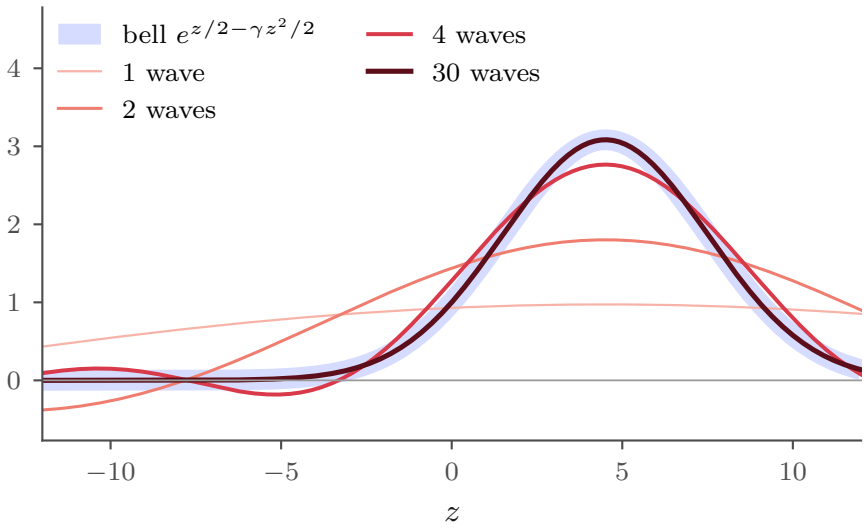


Figure 1.2.: The Hubbard-Stratonovich identity graphically. A bell from Figure 1.1 represented as an average of waves $e^{-\frac{\gamma}{2}(z-c)^2}$.

1.1.5. Arriving at the Characteristic Function

To continue, we want the expectation over x^m to be the innermost one, so we swap the order of the expectations:

$$p(y \mid x^o) = \frac{1}{2}\mathbb{E}_{\gamma\sim\mathrm{PG}(1,0)}\mathbb{E}_{u\sim\mathcal{N}(0,\gamma)}\left[\exp\left(\frac{1}{8\gamma}\right)\mathbb{E}_{p(x^m|x^o)}\left[\exp\left(-iu\left(z-c\right)\right)\right]\right].$$

This is allowed because ...

The innermost expectation is now only over x^m, with x^o fixed, so we split $z = w^T x$ into its observed and missing parts:

$$z = w^T x = w_o^T x^o + w_m^T x^m.$$

The x^o part and c are constant under $\mathbb{E}_{p(x^m|x^o)}$ and factor out:

$$\begin{aligned}\mathbb{E}_{p(x^m|x^o)}\left[\exp\left(-iu\left(z-c\right)\right)\right] &= \mathbb{E}_{p(x^m|x^o)}\left[\exp\left(-iu\left(w_o^T x^o - c + w_m^T x^m\right)\right)\right] \\ &= \exp\left(iu\left(c - w_o^T x^o\right)\right)\cdot\mathbb{E}_{p(x^m|x^o)}\left[\exp\left(-iw_m^T x^m u\right)\right]\end{aligned}$$

This remaining expectation is a known quantity: the characteristic function of a random variable X, $\phi_X(s) := \mathbb{E}\left[\exp(is^T X)\right]$. Writing $-iw_m^T x^m u = i(-uw_m)^T x^m$, we

get:

$$\mathbb{E}_{p(x^m|x^o)}\left[\exp\left(-iu\left(z-c\right)\right)\right] = \exp\left(iu\left(c - w_o^T x^o\right)\right) \cdot \phi_{x^m|x^o}(-uw_m).$$

Intuitively, the characteristic function gives the expectation of a wave under the distribution $p(z)$ of $z = w^T x$: for every frequency u, it gives the signed area under the wave times $p(z)$ (see Appendix A.2 for the derivation). Figure 1.3 shows this for two different waves, i.e. different values of u.

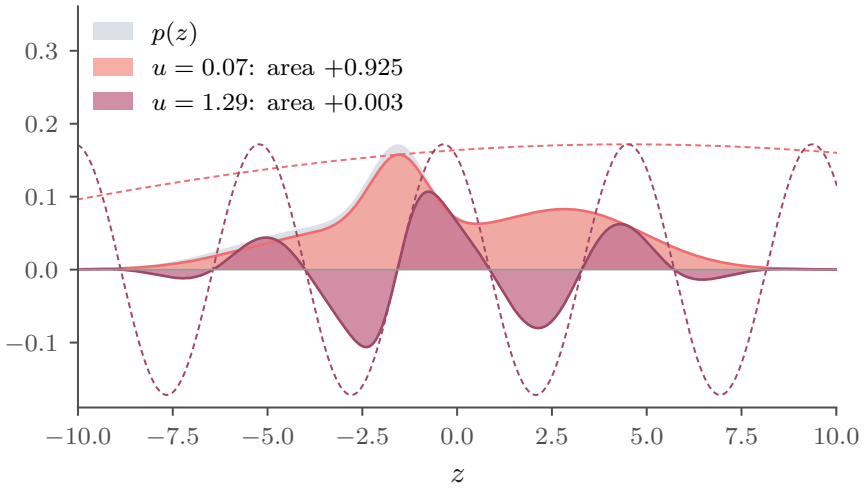


Figure 1.3.: The characteristic function graphically. The expectation of a wave under $p(z)$ is the signed area under the wave times $p(z)$ (shaded). Fast waves average to zero, while slower waves give a non-zero value.

1.1.6. Full Formula

Plugging the result of the previous subsection into $p(y \mid x^o)$ and writing out $c = \frac{1}{2\gamma}$, we obtain:

$$\begin{aligned} p(y \mid x^o) &= \frac{1}{2}\mathbb{E}_{p(x^m|x^o)}\mathbb{E}_{\gamma\sim\mathrm{PG}(1,0)}\left[\exp\left(\frac{1}{8\gamma}\right)\mathbb{E}_{u\sim\mathcal{N}(0,\gamma)}\left[\exp\left(-iu\left(z-c\right)\right)\right]\right] \\ &= \frac{1}{2}\mathbb{E}_{\gamma\sim\mathrm{PG}(1,0)}\mathbb{E}_{u\sim\mathcal{N}(0,\gamma)}\left[\exp\left(\frac{1}{8\gamma}\right)\exp\left(iu\left(\frac{1}{2\gamma} - w_o^T x^o\right)\right)\phi_{x^m|x^o}(-uw_m)\right] \end{aligned}$$

We propose this as a final expression that we can tractably evaluate.

1.1.7. Summary

The expected prediction $p(y \mid x^o)$ is the area under $\sigma(z)\, p(z)$. We replace the sigmoid function by a sum of weighted bells. We get the area under each weighted bell times $p(z)$, by replacing each bell with a sum of waves and put each wave frequency into the characteristic function, which gives the signed area under the wave times $p(z)$.

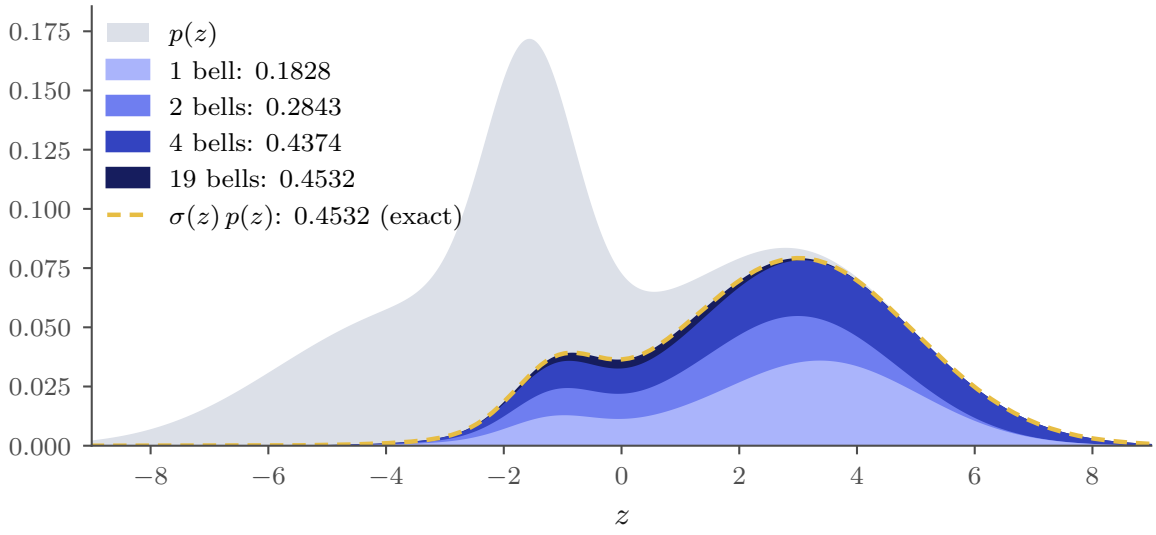


Figure 1.4.: The expected prediction as a sum of bells. $p(y \mid x^o)$ is the area under $\sigma(z)\, p(z)$ (dashed). Adding up the areas under the weighted bells times $p(z)$ (shaded), each computed from the characteristic function, recovers it.

1.2. Calculating the Two Expectations

After the two tricks, $p(y \mid x^o)$ is an expectation over two variables, γ and u. There is no closed form solution for this, so we need to approximate it. There are multiple ways to do this, but in all the principle is the same: we take a set of points (nodes) and associated weights, and replace the expectation with a weighted sum.

We write $f(u, \gamma)$ for the term inside the expectations,

$$f(u,\gamma) := \exp\left(\frac{1}{8\gamma}\right) \exp\left(iu\left(\frac{1}{2\gamma} - w_o^T x^o\right)\right) \phi_{x^m|x^o}(-uw_m).$$

1.2.1. Monte Carlo

The simplest way is to sample both expectations. We draw M samples $\gamma_j \sim \mathrm{PG}(1, 0)$, and for every γ_j we draw N samples $u_{ij} \sim \mathcal{N}(0, \gamma_j)$: Each sample is weighted equally

here, so the weights are $w_j = \frac{1}{M}$ and $\omega_{ij} = \frac{1}{N}$. Since they are not dependent on the sampled values, we can write them outside the sums. The resulting estimator is

$$p(y \mid x^o) \approx \frac{1}{2} \cdot \frac{1}{M} \sum_{j=1}^{M} \frac{1}{N} \sum_{i=1}^{N} f(u_{ij}, \gamma_j).$$

This is always unbiased, but the variance can be quite high and we need a large number of samples to get a good estimate.

1.2.2. Numerical Integration

Both expectations can also be computed by quadrature / numerical integration. The idea is, that instead of having to choose many points and weights randomly to get an acceptable error, we strategically choose a fixed set of specific points and weights that guarantees a certain level of accuracy.

Inner Expectation: Gauss–Hermite

The inner expectation is over $u \sim \mathcal{N}(0, \gamma)$. This is an exact case for a standard Gauss–Hermite quadrature (Section **??**).

(Should this be explained in more detail?) We can easily get the nodes t_i and weights ω_i with `numpy.polynomial.hermite.hermgauss(N)`, which returns the N nodes and weights for the standard normal distribution $\mathcal{N}(0, 1)$. We then scale the nodes by $\sqrt{2\gamma}$ and the weights by $\frac{1}{\sqrt{\pi}}$ to transform them for our distribution.

$$\mathbb{E}_{u \sim \mathcal{N}(0,\gamma)}[f(u, \gamma)] \approx \frac{1}{\sqrt{\pi}} \sum_{i=1}^{N} \omega_i \, f\left(\sqrt{2\gamma}\, t_i, \gamma\right).$$

Plugging this into the full formula gives

$$p(y \mid x^o) \approx \frac{1}{2\sqrt{\pi}} \mathbb{E}_{\gamma \sim \mathrm{PG}(1,0)} \left[\sum_{i=1}^{N} \omega_i \, f\left(\sqrt{2\gamma}\, t_i, \gamma\right) \right].$$

Outer Expectation: Gauss–PG

While for the inner expectation we can use a standard and widely known quadrature rule, there is no standard quadrature rule for an expectation over $\gamma \sim \mathrm{PG}(1, 0)$. So we need to construct one ourselves. We do this by using the Golub–Welsch algorithm

(Section **??**), which is the default choice to construct quadrature nodes and weights for Gauss quadratures.

The Golub–Welsch algorithm only needs the moments of the distribution. For PG$(1, 0)$ they follow from its Laplace transform $\mathcal{L}(t) := \mathbb{E}[e^{-t\gamma}] = \operatorname{sech}(\sqrt{t/2})$, which is known in closed form. Then the k-th moment is, the k-th derivative at zero, up to its sign:

$$\mu_k := \mathbb{E}[\gamma^k] = (-1)^k \, \mathcal{L}^{(k)}(0).$$

We then apply the Golub–Welsch procedure and call the rule *Gauss–PG*. It is shown in Algorithm 1.

Algorithm 1 Gauss–PG quadrature rule

Input: number of nodes n

compute with 200 digits
for $k = 0, \ldots, 2n-1$ **do**
    $\mu_k \leftarrow (-1)^k \, \mathcal{L}^{(k)}(0)$ with $\mathcal{L}(t) = \operatorname{sech}(\sqrt{t/2})$
end for
$H_{ij} \leftarrow \mu_{i+j}$ and $H'_{ij} \leftarrow \mu_{i+j+1}$ for $i, j = 0, \ldots, n-1$
$R \leftarrow$ Cholesky factor of $H = R^T R$
$J \leftarrow R^{-T} H' R^{-1}$
$V \Lambda V^T \leftarrow$ eigendecomposition of J
$\gamma_j \leftarrow \lambda_j$ and $w_j \leftarrow v_{0j}^2$ for $j = 1, \ldots, n$
return $(\gamma_j, w_j)_{j=1}^n$

We do the computation with 200 digits, because the moments only differ very very slightly which makes the Cholesky decomposition fail. For regular floating point precision, the moments are already indistinguishable from about $n \geq 12$.

Since $\mathbb{E}[e^{-\gamma z^2/2}] = \mathcal{L}(z^2/2)$, a rule with n nodes (γ_j, w_j) approximates the sigmoid by a sum of bells,

$$\sigma(z) = \tfrac{1}{2} \, e^{z/2} \, \mathcal{L}\Big(\tfrac{z^2}{2}\Big) \approx \tfrac{1}{2} \, e^{z/2} \sum_{j=1}^{n} w_j \, e^{-\gamma_j z^2/2}. \tag{1.5}$$

Gauss–PG keeps its nodes away from $\gamma = 0$, so the factor $e^{1/(8\gamma_j)}$ in f stays small. Rules with nodes closer to zero can approximate the sigmoid *alone* more accurately, but fail with PCPG, becasue the factor blows up their error. These and other rules for the outer integral are described in Appendix A.3.

Combining the Gauss–PG nodes $(\gamma_j, w_j)_{j=1}^M$ with the Gauss–Hermite nodes $(t_i, \omega_i)_{i=1}^N$

yields the fully deterministic approximation

$$p(y \mid x^o) \approx \frac{1}{2\sqrt{\pi}} \sum_{j=1}^{M} w_j \sum_{i=1}^{N} \omega_i \, f\Big(\sqrt{2\gamma_j}\, t_i, \gamma_j\Big) \,.$$

Sign Flip

Like it is somewhat visible in 1.1, for $z < 0$ the bells approximate the sigmoid quite well, even a small number of bells. For $z > 0$ it becomes harder and for larger z, i.e. $z > 10$, the sigmoid is very close to 1, whereas the sum of bells, no matter how many, tends towards 0 eventually. So we'd have a constant error of 1 for large z. More detail in (Appendix A.5).

To mitigate this we use that the sigmoid is point symmetric around $(0, \frac{1}{2})$:

$$\sigma(-z) = \frac{1}{1+e^z} = \frac{e^{-z}}{e^{-z}+1} = 1 - \sigma(z).$$

For $z > 0$ we can therefore compute $1 - \hat{\sigma}(-z)$ instead of $\hat{\sigma}(z)$. Both use the same nodes and the same approximation of $\mathcal{L}(z^2/2)$. The error at z becomes the error at $-z$. In practice this reduces the error over $|z| \leq 30$ of Gauss–PG with 20 nodes from about 1 to $6 \cdot 10^{-7}$.

The symmetry can be used for the whole expectation as well,

$$\mathbb{E}\Big[\sigma(w^T x) \mid x^o\Big] = 1 - \mathbb{E}\Big[\sigma(-w^T x) \mid x^o\Big],$$

which means the estimator can run with $-w$ and return one minus the result.

Since $z = w^T x$, with some x missing, is random, wether to flip is decided by the sign of the expected value $\mathbb{E}[w^T x \mid x^o] = w_o^T x^o + w_m^T \mathbb{E}[x^m \mid x^o]$. We do this for every point individually.

Since the expectation $\mathbb{E}[x^m \mid x^o]$ is available in closed form for GMMs and PCs, we can compute it efficiently and it doesn't add much additional cost.

1.3. The Characteristic Function of the Density Model

The only place where the density model is used in PCPG is the conditional characteristic function $\phi_{x^m|x^o}$. In this section we describe how to compute it for the two density models we use, GMMs and PCs.

1.3.1. Gaussian Mixture Models

For a Gaussian $X \sim \mathcal{N}(\mu, \Sigma)$, the characteristic function is known in closed form,

$$\phi_X(s) = \mathbb{E}\left[\exp(is^T X)\right] = \exp\left(is^T\mu - \tfrac{1}{2}s^T\Sigma s\right).$$

A GMM has the density $p(x) = \sum_{k=1}^{K} \pi_k \mathcal{N}(x; \mu_k, \Sigma_k)$. Since the expectation is linear, the characteristic function of a mixture is just a mixture of the characteristic functions of its components,

$$\phi_X(s) = \sum_{k=1}^{K} \pi_k \exp\left(is^T\mu_k - \tfrac{1}{2}s^T\Sigma_k s\right).$$

However, in PCPG we need the conditional characteristic function $\phi_{x^m|x^o}$, so the characteristic function of the missing features given the observed ones. To get it, we first need the conditional density $p(x^m \mid x^o)$, which with diagonal covariances, is the mixture:

$$p(x^m \mid x^o) = \sum_{k=1}^{K} \tilde{\pi}_k(x^o)\, \mathcal{N}(x^m; \mu_k^m, \Sigma_k^m),$$

with the responsibilities

$$\tilde{\pi}_k(x^o) = p(k \mid x^o) = \frac{\pi_k \mathcal{N}(x^o; \mu_k^o, \Sigma_k^o)}{\sum_{j=1}^{K} \pi_j \mathcal{N}(x^o; \mu_j^o, \Sigma_j^o)}.$$

Its components are the original ones restricted to the missing features, with mean μ_k^m and covariance Σ_k^m. The responsibilities are the posterior weights for each component. They state how well each component explains the observed features.

The conditional characteristic function is then just the characteristic function of the conditional mixture, so

$$\phi_{x^m|x^o}(s) = \sum_{k=1}^{K} \tilde{\pi}_k(x^o) \exp\left(is^T\mu_k^m - \tfrac{1}{2}s^T\Sigma_k^m s\right).$$

We compute this for every point x^o once and reuse it for all frequencies s, since the responsibilities $\tilde{\pi}_k(x^o)$ only depend on the observed features. This is shown in algorithm 2.

Algorithm 2 Conditional characteristic function of a GMM

Input: GMM parameters $\{\pi_k, \mu_k, \Sigma_k\}_{k=1}^K$ with diagonal $\Sigma_k = \mathrm{diag}(\sigma_k^2)$, partial observation x^o, frequencies $s_1, \ldots, s_L$ over the missing features

for $k = 1, \ldots, K$ **do**
    $\tilde{\pi}_k \leftarrow \dfrac{\pi_k \mathcal{N}(x^o; \mu_k^o, \Sigma_k^o)}{\sum_j \pi_j \mathcal{N}(x^o; \mu_j^o, \Sigma_j^o)}$
end for
for $l = 1, \ldots, L$ **do**
    $\phi_l \leftarrow \sum_{k=1}^K \tilde{\pi}_k \exp\left(i s_l^T \mu_k^m - \frac{1}{2} \sum_{d \in \mathrm{mis}} s_{ld}^2 \sigma_{kd}^2\right)$
end for
return $\phi_1, \ldots, \phi_L$

1.3.2. Probabilistic Circuits

How to compute the CF of a general PC.

PC frameworks usually don't provide a method for it. So we need to implement it ourselves.

The framework SPFlow already offers conditional sampling so we would only need to implement the CF. The framework cirkit doesn't have either.

The PC would also need to be able to give the means of z for the flip.

We could also implement a very simple PC from scratch?

1.4. PCPG Algorithm

Putting everything together leads to the PCPG algorithm shown in Algorithm 3. For one test point, it decides the sign flip, evaluates the density's conditional characteristic function at the frequencies given by the nodes, and returns the weighted sum of f. The Gauss–Hermite and Gauss–PG nodes depend neither on the point nor on the classifier or the density model, so they are computed once beforehand. The density model is only used through its characteristic function, so the algorithm works for every model that can expose one.

Algorithm 3 PCPG

Input: LR weights w, partial observation x^o, density model with conditional characteristic function $\phi_{x^m|x^o}$, Gauss–PG nodes and weights $(\gamma_j, w_j)_{j=1}^M$, Gauss–Hermite nodes and weights $(t_i, \omega_i)_{i=1}^N$

Split $w = (w_o, w_m)$ by the observed/missing mask
flip $\leftarrow \mathbb{E}[w^T x \mid x^o] > 0$
if flip **then**
    $w_o \leftarrow -w_o, \quad w_m \leftarrow -w_m$
end if
$u_{ij} \leftarrow \sqrt{2\gamma_j}\, t_i$ for all i, j
$\phi_{ij} \leftarrow \phi_{x^m|x^o}(-u_{ij}\, w_m)$ for all i, j
$S \leftarrow \sum_{j=1}^M \sum_{i=1}^N w_j\, \omega_i \exp\left(\frac{1}{8\gamma_j}\right) \exp\left(iu_{ij}\left(\frac{1}{2\gamma_j} - w_o^T x^o\right)\right) \phi_{ij}$
$p \leftarrow \frac{1}{2\sqrt{\pi}} \operatorname{Re}(S)$
return $1 - p$ if flip, else p

One can come up with other variants of PCPG by changing how the two expectations are computed. The inner, the outer, or both could be sampled instead. For both also different quadrature rules could be used. Appendix A.4 lists different combinations. In the rest of this thesis, *PCPG* refers to Algorithm 3, and *PCPG-MC* to the variant that samples both expectations.

1.5. Training with PCPG

Up to now, the classifier w was trained on complete data and features were only missing at test time. However, in practice, training data could also have missing features.

In this section we show how PCPG can also be used in training.

1.5.1. Fitting the density

The density model $p(x)$ is fit first. Here PCPG is not needed, and in general this is a standard and well-established procedure. The density is fit, by maximizing the marginal likelihood of the observed features x^o:

$$p(x^o) = \int p(x = (x^o, x^m))\, \mathrm{d}x^m.$$

In general this integral is hard, but can be computed tractably for a GMM or a PC.

For a GMM, we can see this one component k at a time. A component is a Gaussian $\mathcal{N}(\mu_k, \Sigma_k)$, and since our GMM uses diagonal covariances $\Sigma_k = \text{diag}(\sigma_{k1}^2, \ldots, \sigma_{kD}^2)$, every feature d, within a component, has its own one-dimensional Gaussian with mean μ_{kd} and variance σ_{kd}^2. This means that the features are independent of each other within a component. Since the density of independent variables is the product of their individual densities, the density of a point x in component k can be written as

$$p(x \mid k) = \prod_{d=1}^{D} \mathcal{N}(x_d; \mu_{kd}, \sigma_{kd}^2).$$

Now when a feature d is missing, integrating over x_d only touches its own term $\mathcal{N}(x_d; \mu_{kd}, \sigma_{kd}^2)$. Because that term is a density, it integrates to one and it can simply be dropped out of the product. Then only the terms of the observed features remain,

$$p(x^o) = \sum_{k=1}^{K} \pi_k \prod_{d \in \text{obs}} \mathcal{N}(x_d; \mu_{kd}, \sigma_{kd}^2).$$

A PC works the same way. Its leaves are the one-dimensional densities of single features, and when the circuit is smooth and decomposable, the integral over a missing feature can be moved down to its leaves. There the leaves of the missing features are simply set to one and the circuit is evaluated as usual.

$p(x)$ is fit once and kept fixed for the remainder of the training process.

1.5.2. Fitting the classifier

When we already have a way to predict with missing data at test time, training the classifier with missing data is actually quite simple. We just use the same prediction method, that we use at test time, namely the expected prediction over all plausible values of the missing features:

$$p_n = p(y_n = 1 \mid x_n^o) = \mathbb{E}_{p(x^m \mid x_n^o)} \left[\sigma(w^T x) \right].$$

As in an standard logistic regression, w should make true labels as likely as possible. This means we want to maximize the likelihood of true labels, i.e. the probability the classifier assigns to them, or equivalently, minimize the negative log-likelihood. This leads to the following loss function:

$$J(w) = -\frac{1}{n_{\text{train}}} \sum_{n=1}^{n_{\text{train}}} \log p(y_n \mid x_n^o) + \frac{1}{2Cn_{\text{train}}} \|w\|^2. \tag{1.6}$$

Here $p(y_n \mid x_n^o)$ means the probability of the true label of point n: it is p_n if $y_n = 1$ and $1 - p_n$ if $y_n = 0$:

$$\log p(y_n \mid x_n^o) = y_n \log p_n + (1 - y_n) \log(1 - p_n).$$

This is the familiar binary cross-entropy.

The second term here is the L_2 penalty of scikit-learn's logistic regression, with $C = 1$. It keeps w from growing without bound and makes the optimum unique.

As mentioned we compute the expected prediction $p(y_n \mid x_n^o)$ with PCPG, and use automatic differentiation to compute the gradient $\nabla_w J(w)$ of the loss function. Then we can use any gradient-based optimizer, like Adam, and find the optimal w.

1.5.3. Algorithm

Algorithm 4 puts both stages together. First, we fit the density model $p_\theta(x)$, then the classifier starts at $w = 0$ and is trained by Adam on minibatches. Since the expected predictions of PCPG can lie slightly outside $[0, 1]$ due to the quadrature error, we clamp them to $[\varepsilon, 1 - \varepsilon]$.

Algorithm 4 Training a logistic regression with PCPG

Input: incomplete training points $(x_n^o, y_n)_{n=1}^{n_{\text{train}}}$, PCPG nodes and weights, penalty C, epochs E, batch size B, learning rate η, ε

Fit the density model: $\theta \leftarrow \arg\max_\theta \sum_n \log p_\theta(x_n^o)$
$w \leftarrow 0$
for $e = 1, \ldots, E$ **do**
    for every minibatch $\mathcal{B}$ of B training points, in random order **do**
        for $n \in \mathcal{B}$ **do**
            $p_n \leftarrow \text{PCPG}(w, x_n^o)$ with Algorithm 3, clamped to $[\varepsilon, 1 - \varepsilon]$
        end for
        $J \leftarrow -\frac{1}{B} \sum_{n \in \mathcal{B}} (y_n \log p_n + (1 - y_n) \log(1 - p_n)) + \frac{1}{2Cn_{\text{train}}} \|w\|^2$
        $g \leftarrow \nabla_w J$
        $w \leftarrow \text{Adam}(w, g, \eta)$
    end for
end for
return w and p_θ

1.6. A Direct Route for Gaussian Mixtures

Although we use GMMs as density models for PCPG throughout, for a GMM the two tricks are actually not needed: the expected prediction can be computed directly. We include them anyway, to demonstrate the general approach in a simpler setting and to be able to compare the results of PCPG with a reference.

In this section we show how the expected prediction can be computed directly for a GMM.

1.6.1. Reducing the Integral to One Dimension

The prediction $\sigma(w^T x)$ does't depend on individual features, but only on the single number

$$z = w^T x = w_o^T x^o + w_m^T x^m.$$

The observed part $w_o^T x^o$ is fixed. Only the missing part is random. So instead of averaging σ over all missing features, we can average it just over z, and that is always a one-dimensional number. This reduces the dimensionality of the integral from the number of missing features to one, which makes it much easier to compute. The only thing we need for this is the distribution $p(z \mid x^o)$.

1.6.2. The Distribution of z

For a GMM, the distribution of z is easy to find. We write the observed part of z as one fixed number $c := w_o^T x^o$, so that

$$z = c + \sum_{d \in \text{mis}} w_d \, x_d.$$

The only uncertain part is the weighted sum of the missing features.

Assume for a moment that we know which component k the point comes from. Then every missing feature is a one-dimensional Gaussian of its own, and the missing features are independent of each other,

$$x_d \mid k \sim \mathcal{N}(\mu_{kd}, \sigma_{kd}^2) \quad \text{for every } d \in \text{mis}.$$

A weighted sum of independent Gaussians is again Gaussian, and adding the fixed number c only shifts it. So within component k, z is Gaussian as well, $z \mid k \sim \mathcal{N}(m_k, v_k)$, and we only need its mean m_k and its variance v_k.

The mean of a sum is the sum of the means, and the mean of $w_d x_d$ is $w_d \mu_{kd}$. So the mean of z is z with every missing feature replaced by its mean,

$$m_k = \mathbb{E}[z \mid k] = c + \sum_{d \in \text{mis}} w_d \, \mathbb{E}[x_d \mid k] = w_o^T x^o + \sum_{d \in \text{mis}} w_d \, \mu_{kd}.$$

For the variance, the fixed number c does not vary and adds nothing. Since the missing features are independent, the variance of their sum is the sum of their variances, and multiplying x_d by w_d multiplies its variance by w_d^2. So

$$v_k = \text{Var}[z \mid k] = \sum_{d \in \text{mis}} \text{Var}[w_d \, x_d \mid k] = \sum_{d \in \text{mis}} w_d^2 \, \sigma_{kd}^2.$$

For example, with $z = 0.5 + 2x_2 - x_3$, $x_2 \mid k \sim \mathcal{N}(1, 1)$ and $x_3 \mid k \sim \mathcal{N}(0, 4)$, we get $m_k = 0.5 + 2 \cdot 1 - 1 \cdot 0 = 2.5$ and $v_k = 2^2 \cdot 1 + (-1)^2 \cdot 4 = 8$.

In reality, we do not know which component the point comes from. Every component gives its own Gaussian for z, and the observed features tell us how likely each component is, through the responsibilities $\tilde{\pi}_k(x^o)$ of Section 1.3.1. The distribution of z is therefore the mixture of these Gaussians, weighted by the responsibilities,

$$p(z \mid x^o) = \sum_{k=1}^{K} \tilde{\pi}_k(x^o) \, \mathcal{N}(z; m_k, v_k).$$

1.6.3. Computing the Expected Prediction

The expected prediction then splits into one average per component,

$$p(y \mid x^o) = \sum_{k=1}^{K} \tilde{\pi}_k(x^o) \, \mathbb{E}_{z \sim \mathcal{N}(m_k, v_k)} \left[\sigma(z) \right].$$

Each of these averages is a one-dimensional integral of the sigmoid against a Gaussian. While it doesn't have a closed form solution, it can be very accurately computed using numerical integration, like Gauss-Hermite quadrature (Section 1.2.2) or adaptive quadrature using `scipy.integrate.quad`.

$$p(y \mid x^o) \approx \frac{1}{\sqrt{\pi}} \sum_{k=1}^{K} \tilde{\pi}_k(x^o) \sum_{i=1}^{N} \omega_i \, \sigma\left(m_k + \sqrt{2 v_k} \, t_i \right).$$

1.6.4. The Inner Expectation in Closed Form

The mixture $p(z \mid x^o)$ can also be used inside PCPG. After the first trick and swapping the two expectations, the expected prediction is

$$p(y \mid x^o) = \frac{1}{2}\mathbb{E}_{\gamma\sim\mathrm{PG}(1,0)}\mathbb{E}_{z\sim p(z|x^o)}\left[\exp\left(\frac{1}{2}z - \frac{1}{2}\gamma z^2\right)\right].$$

PCPG computes the inner expectation over z with the second trick and Gauss–Hermite quadrature. For a GMM it can be computed exactly instead, because the bell $\exp(\frac{1}{2}z - \frac{1}{2}\gamma z^2)$ times a Gaussian density is again a Gaussian, up to a constant.

Within component k we have $z \sim \mathcal{N}(m_k, v_k)$, and we drop the index k for the derivation. Writing out the density, the expectation is

$$\mathbb{E}_{z\sim\mathcal{N}(m,v)}\left[\exp\left(\frac{1}{2}z - \frac{1}{2}\gamma z^2\right)\right] = \frac{1}{\sqrt{2\pi v}}\int_{-\infty}^{\infty}\exp\left(\frac{1}{2}z - \frac{1}{2}\gamma z^2 - \frac{(z-m)^2}{2v}\right)\mathrm{d}z.$$

The exponent is quadratic in z. We collect its terms by powers of z and write $a := \gamma + \frac{1}{v}$ and $b := \frac{1}{2} + \frac{m}{v}$:

$$\begin{aligned}\frac{1}{2}z - \frac{1}{2}\gamma z^2 - \frac{(z-m)^2}{2v} &= -\frac{1}{2}\left(\gamma + \frac{1}{v}\right)z^2 + \left(\frac{1}{2} + \frac{m}{v}\right)z - \frac{m^2}{2v} \\ &= -\frac{1}{2}az^2 + bz - \frac{m^2}{2v} \\ &= -\frac{1}{2}a\left(z - \frac{b}{a}\right)^2 + \frac{b^2}{2a} - \frac{m^2}{2v}.\end{aligned}$$

The last step completes the square, as in the second trick. The last two terms do not depend on z and can be taken out of the integral. What remains is the integral of an unnormalised Gaussian with variance $\frac{1}{a}$, which is $\sqrt{2\pi/a}$. Therefore

$$\begin{aligned}\mathbb{E}_{z\sim\mathcal{N}(m,v)}\left[\exp\left(\frac{1}{2}z - \frac{1}{2}\gamma z^2\right)\right] &= \frac{1}{\sqrt{2\pi v}}\sqrt{\frac{2\pi}{a}}\exp\left(\frac{b^2}{2a} - \frac{m^2}{2v}\right) \\ &= \frac{1}{\sqrt{va}}\exp\left(\frac{b^2}{2a} - \frac{m^2}{2v}\right).\end{aligned}$$

Both parts simplify. In front, $va = \gamma v + 1$. In the exponent, we use $b = \frac{v+2m}{2v}$ and

$a = \frac{1+\gamma v}{v}$:

$$\begin{aligned}\frac{b^2}{2a} - \frac{m^2}{2v} &= \frac{(v+2m)^2}{8v(1+\gamma v)} - \frac{m^2}{2v} \\ &= \frac{(v+2m)^2 - 4m^2(1+\gamma v)}{8v(1+\gamma v)} \\ &= \frac{v^2 + 4vm - 4\gamma v m^2}{8v(1+\gamma v)} \\ &= \frac{v + 4m - 4\gamma m^2}{8(1+\gamma v)}.\end{aligned}$$

With the index k back, the inner expectation of component k is

$$g_k(\gamma) := \mathbb{E}_{z\sim\mathcal{N}(m_k,v_k)}\left[\exp\left(\frac{1}{2}z - \frac{1}{2}\gamma z^2\right)\right] = \frac{1}{\sqrt{1+\gamma v_k}}\exp\left(\frac{v_k + 4m_k - 4\gamma m_k^2}{8(1+\gamma v_k)}\right).$$

As a check, for $\gamma = 0$ this gives $\exp(\frac{1}{2}m_k + \frac{1}{8}v_k)$, which is the known mean $\mathbb{E}[e^{z/2}]$ of a log-normal variable.

The inner expectation over the whole mixture is the weighted sum of the components, and only the outer expectation over γ is left. Computing it with the Gauss–PG nodes and weights $(\gamma_j, w_j)_{j=1}^M$ of Section 1.2.2 gives

$$\begin{aligned}p(y \mid x^o) &= \frac{1}{2}\mathbb{E}_{\gamma\sim\mathrm{PG}(1,0)}\left[\sum_{k=1}^K \tilde{\pi}_k(x^o)\, g_k(\gamma)\right] \\ &\approx \frac{1}{2}\sum_{j=1}^M w_j \sum_{k=1}^K \tilde{\pi}_k(x^o)\, g_k(\gamma_j).\end{aligned}$$

The sign flip of Section 1.2.2 carries over unchanged: replacing w by $-w$ only changes the sign of every m_k, and leaves v_k as it is.

As an estimator this is not useful on its own, since the one-dimensional integrals of the previous subsection are simpler and exact. But it is PCPG with a perfect inner quadrature: the only error left is the error of the Gauss–PG rule. Comparing it with PCPG therefore separates the error of the outer quadrature from the error of the inner one.

1.6.5. Why This Does Not Work for General PCs

The direct route needs two things: that we can go through the components one by one, and that z is Gaussian within each of them. A PC is also a mixture, but the

number of its components grows exponentially with its size, so going through them one by one is only possible for small circuits.

Also a general PC could have non-Gaussian leaves, which would make z not Gaussian within a component.

2. Experimental Results

This chapter presents the experiments: first how accurately PCPG estimates the expected prediction, then how well a logistic regression can be trained with PCPG on incomplete data. The explanations of the results follow in Chapter 3.

2.1. Estimating the Expected Prediction

Setup. The data are generated with scikit-learn's `make_classification`: 10,000 samples, of which 2 features are informative, 2 are linear combinations of these, and all others are independent noise; 8,000 samples are used for training and 2,000 for testing. The density $p(x)$ is a GMM with $K = 4$ diagonal components, the classifier a logistic regression without intercept. The number of features is $d \in \{5, 16, 50, 158, 500, 1581, 5000\}$, a factor $\sqrt{10}$ apart. For every test point a fraction $\rho \in \{0.3, 0.5, 0.8\}$ of the features is hidden at random. MC and PCPG get the same budget of 1,216 evaluations: MC draws 1,216 samples, and PCPG uses $M = 19$ Gauss–PG nodes and $n_H = 64$ Gauss–Hermite nodes. The error is the RMSE over 50 test points against the exact value, which we compute from the Gaussian mixture of $z \mid x^o$ (Section 1.6) with adaptive quadrature. The time is that of one batched call over all 2,000 test points on a GPU.

Default classifier. Figure 2.1 shows the result with scikit-learn's default regularisation ($C = 1$). Up to $d = 158$, PCPG reaches an RMSE between $2 \cdot 10^{-7}$ and $2 \cdot 10^{-5}$, two to four orders of magnitude below MC, whose error stays between $3 \cdot 10^{-3}$ and $1.3 \cdot 10^{-2}$ at every d. From $d = 500$ on, the error of PCPG rises, and at $d = 5000$, with an RMSE between $2 \cdot 10^{-2}$ and $4 \cdot 10^{-2}$, it is worse than MC. The errors of MC and of mean imputation hardly depend on d. At the same time, the norm of the classifier grows with d (Table 2.1): it is about 2.2 up to $d = 158$, and 9.5 at $d = 5000$, where the classifier overfits. Section 3.3 explains why a larger norm makes the expectation harder for PCPG.

Fixed weight norm. To separate the effect of d from that of $\|w\|$, we rescale the weights of the default classifier to unit norm, $w \leftarrow w/\|w\|$, for every d. This keeps the direction of w, and with it every decision of the classifier, and only puts $z = w^T x$ on

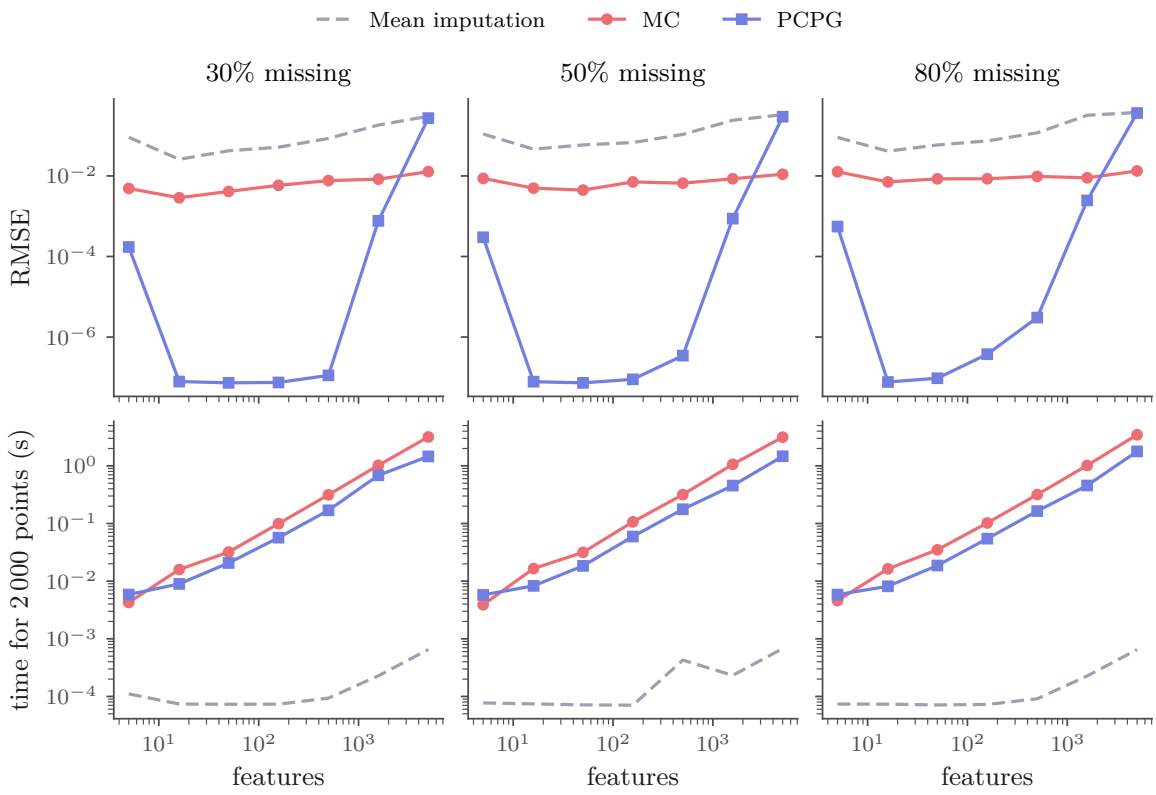


Figure 2.1.: Mean imputation, MC and PCPG against the number of features d, for 30%, 50% and 80% missing features, with the default logistic regression ($C = 1$). Top: RMSE over 50 test points. Bottom: time for all 2,000 test points. MC and PCPG use a budget of 1,216 evaluations.

the same scale for every d. With the norm held fixed (Figure 2.2), the error of PCPG no longer depends on d. It lies between $3.6 \cdot 10^{-8}$ and $6 \cdot 10^{-8}$ for every d and missing rate, including $d = 1581$ and $d = 5000$. MC stays between $2 \cdot 10^{-3}$ and $9 \cdot 10^{-3}$.

Runtime. The bottom rows of both figures are the same, since the time does not depend on w. For all methods it grows linearly in d. From $d = 16$ on, PCPG is about twice as fast as MC at the same budget, for example about 1.4 s against 3.0 s at $d = 5000$.

2.2. Training with Missing Features

In this section we train a logistic regression on incomplete data with PCPG inside the loss (Section 1.5) and compare it with the usual ways of training on incomplete data.

Table 2.1.: The logistic regression with the default regularisation ($C = 1$) for every number of features d: the norm of w, and accuracy and log loss on the 8,000 training and 2,000 test points.

d	$\|w\|$	**train acc.**	**test acc.**	**test log loss**
5	2.2	0.891	0.887	0.312
16	2.3	0.894	0.875	0.314
50	2.3	0.894	0.888	0.314
158	2.4	0.894	0.875	0.311
500	2.7	0.902	0.874	0.337
1581	6.6	0.938	0.817	0.764
5000	9.5	1.000	0.805	1.215

2.2.1. Compared Models

All models are logistic regressions without intercept. They differ only in the training points they see and in the prediction their loss is built from (Table 2.2):

- **Complete data:** an ordinary logistic regression on the complete training points. This is an oracle, since these points are not available when features are missing. It shows what training could reach without missing features.
- **Mean imputation:** an ordinary logistic regression on the training points with every missing feature replaced by the mean of its observed values.
- **MC, PCPG, Exact:** the marginal likelihood (1.6) on the incomplete training points, with the expectation estimated by Monte Carlo with S samples, by deterministic PCPG with $M \times N$ nodes, or computed exactly. Exact uses the direct route for Gaussian mixtures (Section 1.6) with Gauss–Hermite quadrature for every component. It is only available for the GMM, and serves as the target of MC and PCPG.

Model	**Training points**	**Prediction in the loss**
Complete data	complete x	$\sigma(w^T x)$
Mean imputation	x^o, x^m filled by means	$\sigma(w^T x)$
MC	x^o	$\frac{1}{S}\sum_s \sigma(w^T x^{(s)})$
PCPG	x^o	PCPG, $M \times N$ nodes
Exact	x^o	$\sum_k \tilde{\pi}_k(x^o)\mathbb{E}_{\mathcal{N}(m_k,v_k)}[\sigma(z)]$

Table 2.2.: The compared models. The last three minimise the marginal likelihood (1.6), the first two an ordinary logistic regression loss. All use the same L_2 penalty.

MC, PCPG and Exact are trained in the same way, with Adam on minibatches, starting from $w = 0$. Complete data and mean imputation are ordinary logistic regressions and

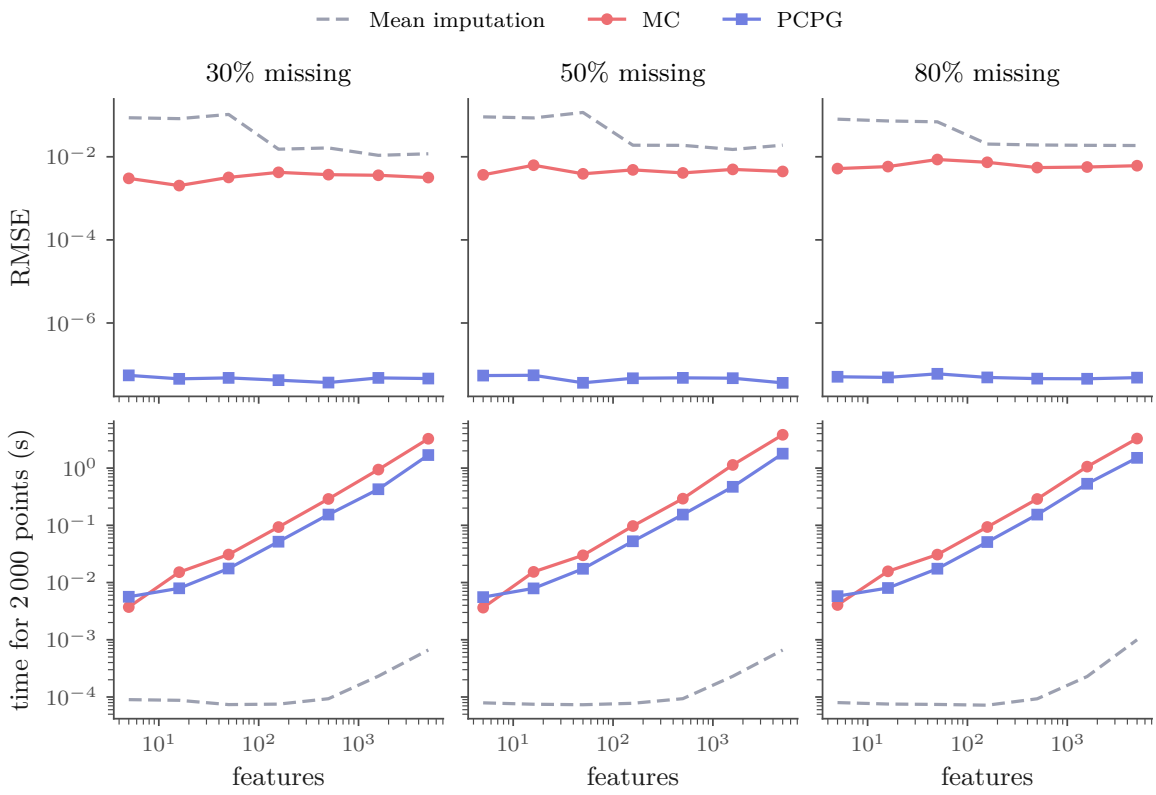


Figure 2.2.: The same comparison as in Figure 2.1, with the weights of the logistic regression rescaled to unit norm, $w/\|w\|$, for every number of features d.

are fit with scikit-learn, which finds their optimum exactly.

2.2.2. Evaluation

Every trained w is scored in three ways.

- **Objective gap** $J(w) - J(w^*)$, where J is the exact marginal likelihood (1.6) on the training points and w^* its optimum, found by full-batch L-BFGS. It measures how well a model minimises the marginal likelihood, so its target is Exact. For complete data and mean imputation it also contains the difference between their objective and the marginal likelihood.
- **Test accuracy on incomplete points:** every model predicts with the exact expected prediction on test points with the same fraction of missing features, so the models only differ in w.
- **Test accuracy on complete points:** the accuracy of $\sigma(w^T x)$ on the complete test points, i.e. how good w is as a classifier.

For both accuracies the target is the complete-data model, the classifier that training could reach if no features were missing.

2.2.3. Setup

The data are again generated with scikit-learn's `make_classification`, with 10,000 samples, 8,000 for training and 2,000 for testing. In contrast to Section 2.1, half of the features are informative and the others are noise, without redundant features: with only two informative features, almost all missing features would be irrelevant for the label. The number of features is $d \in \{5, 10, 20, 50, 100\}$. In every training and test point, $\lfloor \rho d \rfloor$ features are hidden at random, with $\rho \in \{0.3, 0.5, 0.8\}$. The density is a GMM with $K = 4$ diagonal components, fit on the incomplete training points. PCPG uses $M = 10$ Gauss–PG nodes and $N = 32$ Gauss–Hermite nodes, and MC gets the same budget of $S = 320$ samples per point and step. Exact uses 64 Gauss–Hermite nodes per component. Adam runs for 100 epochs with minibatches of 1,024 points and a learning rate of 0.01.

2.2.4. Results

All results are for a single random choice of the hidden features per setting. With 2,000 test points, an accuracy has a standard error of about 0.9 percentage points, so smaller differences in accuracy are not meaningful. The results are discussed in Section 3.4.

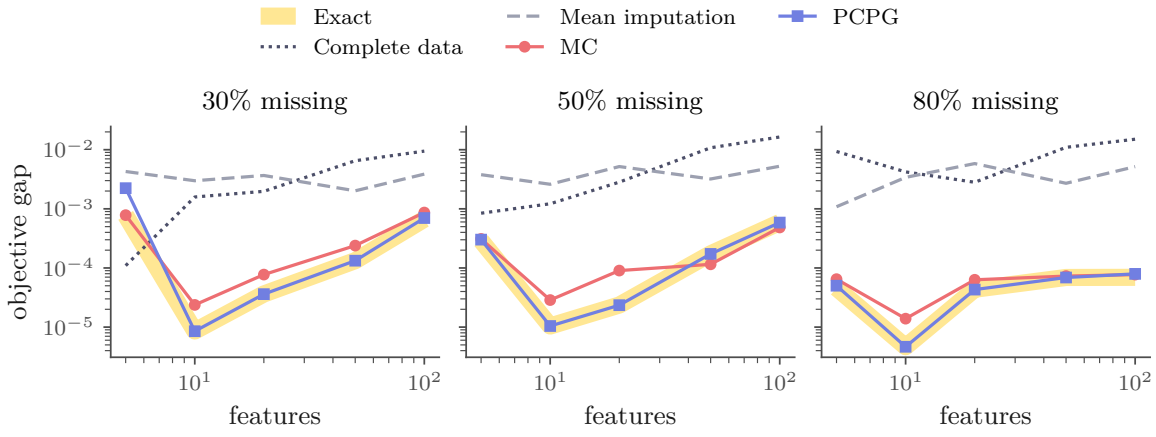


Figure 2.3.: Objective gap $J(w) - J(w^*)$ of the trained classifiers against the number of features d, for 30%, 50% and 80% missing features. J is the exact marginal likelihood (1.6) on the training points, and the target is the classifier trained with the exact expectation (yellow band). MC uses $S = 320$ samples, PCPG 10×32 nodes.

Objective gap. Figure 2.3 shows how well every model minimises the marginal likelihood. MC, PCPG and Exact mostly end between $5 \cdot 10^{-6}$ and 10^{-4} above the optimum, complete data and mean imputation mostly between 10^{-3} and $2 \cdot 10^{-2}$. PCPG follows Exact closely in almost all settings. MC lies slightly above it, by up to $7 \cdot 10^{-5}$ for 10 and 20 features. At both ends of the range of d, the gaps of all three are larger. The only setting in which PCPG falls behind Exact is 5 features with 30% missing, where its gap of $2.2 \cdot 10^{-3}$ is three times that of Exact.

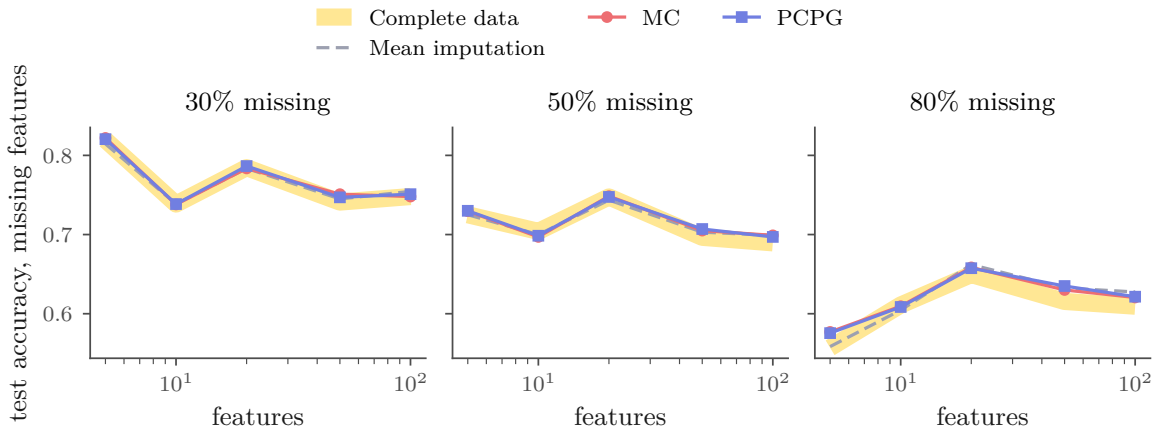


Figure 2.4.: Test accuracy on test points with the same fraction of missing features, every classifier with the exact expected prediction. The target is the classifier trained on complete data (yellow band).

Accuracy on incomplete test points. When the test points are incomplete as well (Figure 2.4), all classifiers reach almost the same accuracy at 30% and 50% missing. At 80% missing, the classifiers trained on the marginal likelihood are more accurate than the one trained on complete data, by 2.0 percentage points at 5 features (57.7% against 55.7%), 1.9 at 50 features and 1.4 at 100 features. Mean imputation lies close to the marginal likelihood at 50 and 100 features, but 1.9 percentage points below it at 5 features.

Accuracy on complete test points. At 30% and 50% missing, all classifiers lie within 1.5 percentage points of the complete-data classifier (Figure 2.5). At 80% missing they fall behind it, by up to 2.9 percentage points at 100 features. Here, the classifiers trained on the marginal likelihood are ahead of mean imputation at 10 and 20 features (81.1% against 79.4% at 20 features), and level with it at 50 and 100 features. At 5 features, mean imputation is ahead by 1.1 percentage points.

Adam against an exact solver. To check that the training by Adam does not distort the comparison, we also fit complete data and mean imputation with Adam, in the

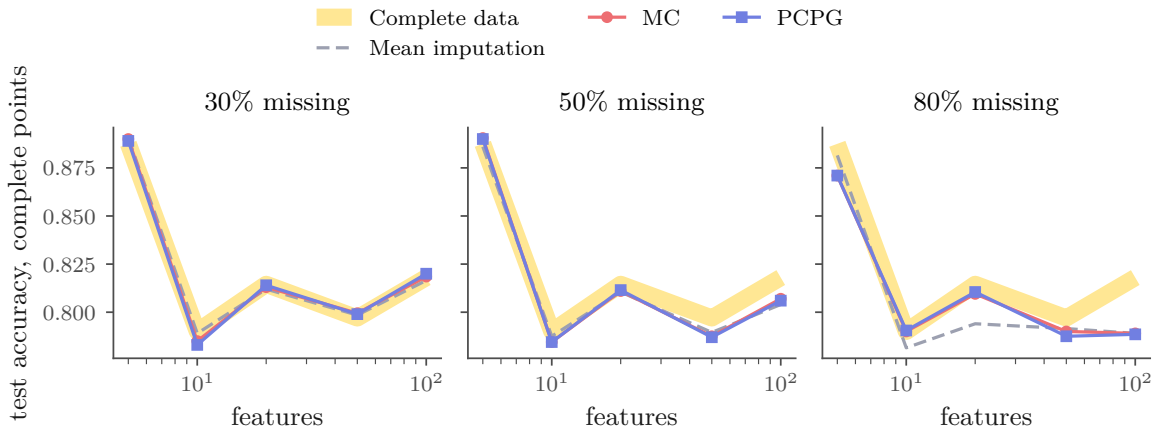


Figure 2.5.: Test accuracy of $\sigma(w^T x)$ on the complete test points, i.e. how good the classifier is when nothing is missing. The target is the classifier trained on complete data (yellow band).

same way as MC, PCPG and Exact (Figure 2.6). Both fits give practically the same accuracies, and the same objective gap except at 5 features, where the gaps differ by up to a factor of five, in both directions.

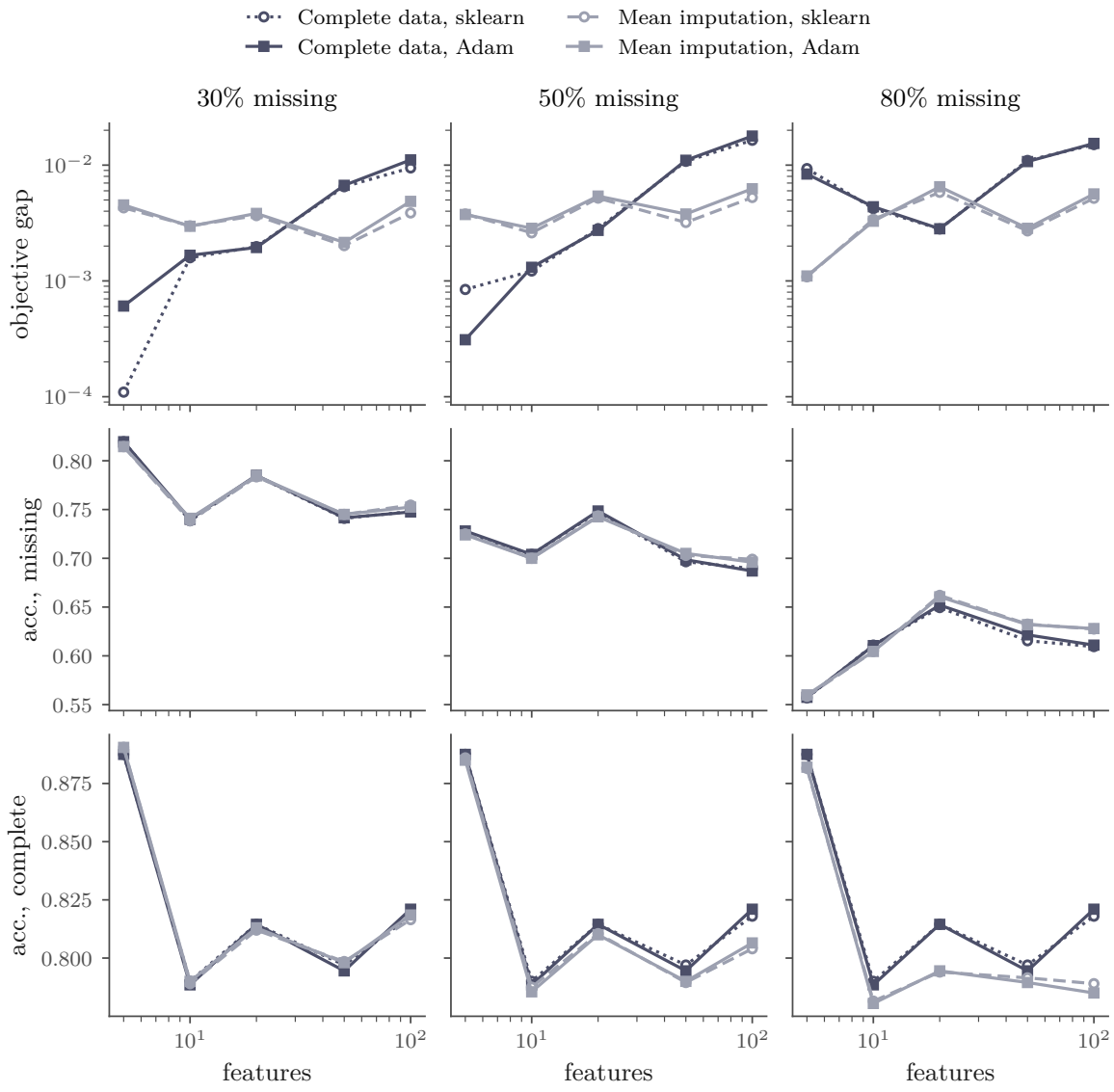


Figure 2.6.: The two ordinary logistic regressions, fit with scikit-learn (open markers) and with the same Adam schedule as the other models (filled markers): objective gap, and test accuracy on incomplete and on complete test points.

3. Discussion

3.1. Bias and Variance

We compare the bias and variance of MC with PCPG-MC, which samples both expectations, and PCPG, which computes both by quadrature.

3.1.1. MC Estimator

The MC estimator draws N samples $x^{m(i)} \sim p(x^m \mid x^o)$ and returns

$$\widehat{p}_{\mathrm{MC}}(N) = \frac{1}{N} \sum_{i=1}^{N} \sigma\Big(w^T x^{(i)}\Big) .$$

Bias. MC is unbiased for all $N \geq 1$: sampling from $p(x^m \mid x^o)$ and averaging $\sigma(w^T x)$ is a direct Monte Carlo estimate of the target expectation, so $\mathbb{E}[\widehat{p}_{\mathrm{MC}}(N)] = p(y \mid x^o)$ exactly.

Variance. The draws are i.i.d., so the variance of their average is the per-sample variance $\sigma^2_{\mathrm{MC}} := \mathrm{Var}_{p(x^m|x^o)}\Big[\sigma(w^T x)\Big]$ divided by N. Since $\sigma(\cdot) \in (0,1)$, the per-sample variance is bounded: for any $[0,1]$-valued random variable Y,

$$\mathrm{Var}(Y) = \mathbb{E}[Y^2] - \mathbb{E}[Y]^2 \leq \mathbb{E}[Y] - \mathbb{E}[Y]^2 = \mathbb{E}[Y](1 - \mathbb{E}[Y]) \leq \tfrac{1}{4},$$

using $Y^2 \leq Y$ on $[0,1]$ and $p(1-p) \leq 1/4$. Therefore

$$\mathrm{Var}[\widehat{p}_{\mathrm{MC}}(N)] = \frac{\sigma^2_{\mathrm{MC}}}{N} \leq \frac{1}{4N}.$$

3.1.2. PCPG-MC Estimator

PCPG-MC draws M outer samples $\gamma_j \sim \mathrm{PG}(1,0)$ and, for each of them, L inner samples $u_{ij} \sim \mathcal{N}(0,\gamma_j)$ (Section 1.2.1). With $Z := \frac{1}{2}\mathrm{Re}[f(u,\gamma)]$ for a single sample, it returns

$$\widehat{p}_{\text{PCPG-MC}}(M,L) = \frac{1}{ML}\sum_{j=1}^{M}\sum_{i=1}^{L} Z_{ij}.$$

Bias. PCPG-MC is unbiased as well. Both expectations are approximated by ordinary MC, so by linearity of expectation the estimator equals the true double integral, which is $p(y \mid x^o)$ by the PG and Hubbard–Stratonovich identities.

Output range. Since $|\phi_{x^m|x^o}| \le 1$, a single sample satisfies $|Z| \le \frac{1}{2}e^{1/(8\gamma)}$. Because $\gamma \sim \mathrm{PG}(1,0)$ can be arbitrarily small, this bound is unbounded, so individual PCPG samples are not confined to $[0,1]$ and can be negative or exceed 1, while every MC sample lies in $(0,1)$.

Variance. The estimator has two nested levels of randomness. For a fixed γ_j, the average of its L inner samples has the conditional variance $\sigma^2_{A,\gamma_j}/L$, with $\sigma^2_{A,\gamma} := \mathrm{Var}_{u|\gamma}[Z \mid \gamma]$. The law of total variance over both levels and the M independent outer samples give

$$\mathrm{Var}[\widehat{p}_{\text{PCPG-MC}}(M,L)] = \frac{\sigma_A^2}{ML} + \frac{\sigma_B^2}{M}, \qquad \sigma_A^2 := \mathbb{E}_\gamma\left[\sigma^2_{A,\gamma}\right], \quad \sigma_B^2 := \mathrm{Var}_\gamma\left[\mathbb{E}_{u|\gamma}[Z \mid \gamma]\right].$$

σ_A^2 measures how much the inner expectation fluctuates for a typical γ, and σ_B^2 how much γ itself shifts the result.

The inner variance is infinite. The problem comes from completing the square in Trick 2, which introduced the factor $e^{1/(8\gamma)}$. For a fixed z, the inner expectation is known exactly from the HS identity,

$$\mathbb{E}_{u\sim\mathcal{N}(0,\gamma)}\left[e^{-iu(z-c)}\right] = \exp\left(-\tfrac{1}{2}\gamma\,(z-c)^2\right) \approx e^{-1/(8\gamma)} \quad \text{for small } \gamma,$$

which is tiny. Multiplied by $e^{1/(8\gamma)}$ it gives a value of order one. So the inner average has to produce a tiny number from terms of modulus one that almost cancel, and every bit of noise in that average is amplified by $e^{1/(8\gamma)}$, which is already about $2.7\cdot 10^5$ for $\gamma = 0.01$. In terms of variance, a single inner sample has the second moment $e^{1/(4\gamma)}$. Near zero the density of $\mathrm{PG}(1,0)$ decays only like $e^{-1/(8\gamma)}$, which is not fast enough to compensate, so averaged over γ the second moment is infinite: formally $\sigma_A^2 = \infty$.

Small values of γ are rare, which is why the estimator still works in practice, but when they occur they produce extreme samples.

The outer variance shrinks with more missing features. For a GMM with diagonal covariances, the characteristic function of the missing features satisfies

$$\left|\phi_{x^m|x^o}(-uw_m)\right| \le \exp\left(-\frac{u^2}{2}\,D_m\right), \qquad D_m := \min_k \sum_{j\in\text{mis}} w_j^2\,\sigma_{kj}^2,$$

where D_m is the variance of the missing part of $w^T x$. When D_m is large, the integrand is nearly zero unless $|u|$ is small, so the inner expectation is close to zero for a wide range of γ and hardly varies with it: $\sigma_B^2 \approx 0$. More missing features increase D_m, so PCPG-MC gains relative to MC as the missing rate grows, while MC behaves the opposite way, since more missing features make $w^T x$ more random and increase σ_{MC}^2.

3.1.3. PCPG Estimator

Variance. Replacing both expectations with quadrature (Gauss–PG for the outer, Gauss–Hermite for the inner) makes the estimator fully deterministic:

$$\text{Var}[\widehat{p}_{\text{PCPG}}] = 0.$$

Its output is bounded, since the nodes γ_j are fixed positive numbers and the factor $e^{1/(8\gamma_j)}$ is a known finite constant for each of them. The amplification is mild, because the smallest Gauss–PG node is not close to zero ($\gamma_{\min} \approx 0.046$ for 19 nodes, giving $e^{1/(8\gamma_{\min})} \approx 15$).

Bias. Both expectations introduce a quadrature error. Gauss–Hermite with N nodes is exact for polynomials in u up to degree $2N-1$, and for analytic integrands its error decays super-algebraically in N. The Gauss–PG rule with M nodes is exact for polynomials in γ up to degree $2M-1$. At $M = 19$ and $N \ge 20$, both errors are numerically negligible for moderate $|w^T x|$. For large positive $w^T x$ the outer error is amplified by $e^{z/2}$, which the sign flip (Section 1.2.2) removes; with the flip, the outer rules fitted directly to the sigmoid (Appendix A.3.2) are more accurate than Gauss–PG with fewer nodes. The inner Gauss–Hermite error for large $|w^T x|$ is not affected by the flip. The infinite inner variance of PCPG-MC does not disappear completely, however; it turns into quadrature error. With $u = \sqrt{2\gamma}\,t$ the inner integrand contains the factor $\exp(-i(z-c)\sqrt{2\gamma}\,t)$, which oscillates with the frequency $|z-c|\sqrt{2\gamma}$ in the Hermite variable t. This frequency is large when γ is small, but also when $|w^T x|$ is

large. A fixed number of nodes cannot resolve fast oscillations, and the exact inner value it should produce is then tiny, so the rule has to sum terms of modulus one to almost zero, and any error in that sum is multiplied by $e^{1/(8\gamma)}$. The damping of the characteristic function helps when many features are missing, because it suppresses large $|u|$; with few missing features and a confident prediction, the inner quadrature error can become significant.

3.1.4. Comparison

Table 3.1 summarises the properties of the three estimators.

Table 3.1.: Summary of the three estimators. MC uses N samples, PCPG-MC M outer and L inner samples, PCPG M Gauss–PG and N Gauss–Hermite nodes. "CF" means the estimator requires evaluating $\phi_{x^m|x^o}$ rather than drawing samples from $p(x^m \mid x^o)$.

Estimator	**Variance**	**Bias**	**Needs**
MC	$\sigma^2_{\mathrm{MC}}/N \leq 1/(4N)$	0	samples
PCPG-MC	$\sigma_A^2/(ML) + \sigma_B^2/M$, with $\sigma_A^2 = \infty$	0	CF
PCPG	0	quadrature error	CF

Bias. MC and PCPG-MC are exactly unbiased. PCPG introduces a quadrature error, which decays fast in the number of nodes and is negligible at the parameter choices used ($N \geq 20$ Gauss–Hermite and $M = 19$ Gauss–PG nodes), as long as $|w^T x|$ is moderate.

Variance. MC has the variance σ^2_{MC}/N, bounded by $1/(4N)$. PCPG-MC is unbiased as well, but its inner variance σ_A^2 is formally infinite because of the factor $e^{1/(8\gamma)}$, so rare small values of γ produce extreme samples, and its outer variance σ_B^2 only becomes small when many features are missing. PCPG has no variance at all: its error is pure bias from the quadrature.

When to use which. MC is the simplest choice when sampling from $p(x^m \mid x^o)$ is easy. PCPG-MC mainly serves as a reference: it shows that the identities behind PCPG are exact, but sampling the two expectations is not competitive. PCPG is the method we propose: it only needs the characteristic function, and it is deterministic, so repeated evaluations at the same point agree exactly and it can be differentiated, for example when training through it.

3.2. Effect of Dimensionality

Two distinct quantities can vary: the number of missing features d_m at fixed total dimension d (equivalently, the missing ratio $\rho = d_m/d$), and the total dimension d itself at a fixed ratio ρ. Their effects are different and we treat them separately. The key quantity that links both estimators is the *missing-feature energy*

$$D_m := \sum_{j \in \text{mis}} w_j^2 \, \sigma_{kj}^2,$$

which measures how much variance the uncertain part of $w^T x$ contributes.

Effect of the missing ratio (more missing features, fixed d). As d_m grows at fixed d, more terms enter D_m, so D_m increases.

For *MC*, the uncertain part of the dot product is $w_m^T x^m$, whose variance under $p(x^m \mid x^o)$ is exactly D_m. A larger D_m spreads the distribution of $\sigma(w^T x)$ more widely, pushing $\sigma^2_{\text{MC}} = \text{Var}[\sigma(w^T x)]$ toward its upper bound of $1/4$. *MC variance increases with the missing ratio.*

For *PCPG*, the key observation is what the CF value at a frequency actually means. The CF $\phi_X(s) = \mathbb{E}[e^{isX}]$ has modulus at most 1. Intuitively, $|e^{isX}| = 1$ for every realisation, but the complex exponential e^{isX} rotates in the complex plane as X varies; when X is widely spread, these rotations point in different directions and cancel when averaged, driving $|\phi_X(s)|$ toward zero. Concretely, if X has variance σ_X^2, then for large $|s|$ the CF decays at least as fast as $e^{-\sigma_X^2 s^2/2}$; a larger variance means faster decay.

In the PCPG integral, the CF is evaluated at the frequency $s = -uw_m$, which is proportional to the inner sample u. The relevant variance is that of the projection $w_m^T x^m$ onto the classifier direction, which equals D_m. Hence

$$\left|\phi_{x^m|x^o}(-uw_m)\right| \;\le\; \exp\!\left(-\tfrac{1}{2}u^2 \, D_m\right).$$

When D_m is large, this bound drops to nearly zero already for moderate $|u|$ — the integrand $h(\gamma, u)$ is effectively zero outside a narrow band $|u| \lesssim 1/\sqrt{D_m}$ around zero. Larger D_m makes this band narrower: "stronger CF damping" means the PCPG integrand is non-negligible only for very small u.

Two consequences follow from a narrow effective integration range. First, σ_A^2 decreases: the $e^{1/(8\gamma)}$ blowup that drives inner variance is large only when γ is small, which makes $u \sim \mathcal{N}(0, \gamma)$ small as well. In this regime the CF is near 1 regardless of D_m, so the blowup is not directly suppressed. However, for large γ — where most PG mass lies — the Gaussian $\mathcal{N}(0, \gamma)$ places u well outside the narrow band, and the CF kills those samples. The net effect is fewer extreme inner samples, lowering σ_A^2. Second, σ_B^2

decreases: the inner expected value $\mathbb{E}_{u|\gamma}[Z \mid \gamma]$ is near zero for any γ large enough that $\sqrt{\gamma} \gg 1/\sqrt{D_m}$ (essentially all large PG draws), and near 1/2 only for the rare very small γ draws. As D_m grows, the threshold shifts to smaller γ and a larger fraction of PG draws gives $\mathbb{E}[Z|\gamma] \approx 0$, so the variation of this function over γ — which is σ_B^2 — shrinks. *The variance of PCPG-MC decreases with the missing ratio.*

The same narrowing of the integrand reduces the Gauss–Hermite quadrature error for PCPG: a more concentrated integrand is easier to integrate accurately with a fixed number of nodes, so the bias also decreases with ρ.

Effect of total dimension (fixed missing ratio ρ). As d grows with ρ fixed, $d_m = \rho d$ grows, but so does $d_o = (1-\rho)d$. Each individual weight scales roughly as $w_j = O(1/\sqrt{d})$ in a well-fit logistic regression (the discriminative signal is spread over more features), so each term in D_m is $O(1/d)$. Summing over $d_m = \rho d$ missing features gives $D_m = O(\rho)$: bounded, regardless of d.

For *MC*, the variance of $w_m^T x^m$ stays at $D_m = O(\rho)$, so the distribution of $\sigma(w^T x)$ approaches a fixed limit as $d \to \infty$ and σ^2_{MC} neither grows nor shrinks.

For *PCPG*, D_m stays bounded, so the CF damping does not improve and the variances σ_A^2, σ_B^2 stay roughly constant.

Higher total dimension at fixed missing ratio does not improve PCPG over MC. The relative advantage of PCPG depends on the missing ratio ρ, not on the total number of features d alone.

Summary. As the missing ratio ρ grows: MC variance increases toward $1/(4N)$, while PCPG variance (both σ_A^2 and σ_B^2) decreases due to CF concentration. As total dimension d grows at fixed ρ: both estimators are largely unaffected, since D_m stays bounded. The relative advantage of PCPG over MC is therefore tied to the missing ratio, and is strongest in the high-missing-data regime where accurate marginalisation matters most.

3.3. Effect of the Classifier's Weight Norm

The argument of Section 3.2 for a growing total dimension d assumes a well-fitted classifier, whose individual weights shrink like $O(1/\sqrt{d})$, so that $z = w^T x$ stays on the same scale. The experiments of Section 2.1 show that this assumption matters: with the default regularisation, the norm of w grows with d, and PCPG with a fixed number of nodes loses accuracy at high dimensions. When the norm of w is held fixed, the effect disappears.

Cause: the classifier overfits. Only four features carry information about the label, so every additional feature is noise. With $C = 1$ the logistic regression fits this noise once there are many features compared to the 8,000 training points (Table 2.1). The norm of w grows from about 2.2 up to $d = 158$ to 2.7, 6.6 and 9.5 at $d = 500$, 1581 and 5000. At $d = 5000$ the classifier reaches 100% training accuracy, but only 80.5% test accuracy, with a test log loss of 1.22. A larger w makes $z = w^T x$ larger. At $d = 5000$, the median of $|m_k|$ over all test points and components is between 8.6 and 14.9 (depending on the missing rate), and the median standard deviation $\sqrt{v_k}$ between 2.8 and 4.8. At $d = 16$ they are between 2.4 and 2.7, and between 0.5 and 1.2.

Why a larger z is harder for PCPG. Consider one component of the mixture, $z \mid k \sim \mathcal{N}(m_k, v_k)$, and substitute $u = \sqrt{2\gamma}\, t$ as in Section 1.2.2. The inner integrand of this component becomes

$$e^{1/(8\gamma)}\, e^{\,iat - bt^2}, \qquad a = \frac{1}{\sqrt{2\gamma}} - \sqrt{2\gamma}\, m_k, \qquad b = \gamma\, v_k,$$

and Gauss–Hermite has to approximate

$$\int_{-\infty}^{\infty} e^{-t^2}\, e^{\,iat - bt^2}\, \mathrm{d}t = \sqrt{\frac{\pi}{1+b}}\, \exp\!\left(-\frac{a^2}{4(1+b)}\right).$$

For large $|a|$ this value is exponentially small, but every term of the quadrature sum has a modulus of up to ω_i. The sum therefore has to cancel almost completely, which a fixed rule can only do if its nodes resolve the oscillation e^{iat}. The n_H nodes lie within roughly $\pm\sqrt{2n_H}$ with a spacing of about $\pi/\sqrt{2n_H}$, which gives the rule of thumb $n_H \gtrsim a^2/2$. Whatever error remains is then multiplied by $e^{1/(8\gamma)}$.

Because of the flip (Section 1.2.2), m_k is mostly negative, so $a = 1/\sqrt{2\gamma} + \sqrt{2\gamma}\,|m_k|$ grows linearly in $|m_k|$, and thereby in $\|w\|$. The 19 Gauss–PG nodes reach up to $\gamma \approx 12.8$. There, $|m_k| = 2.4$ gives $a \approx 12$, which 64 nodes still roughly resolve, while $|m_k| = 10$ gives $a \approx 50$, far more than 64 nodes can handle. The largest nodes carry small weights, so the rule of thumb is only a rough guide, but it explains the trend. The second parameter b grows with $\|w\|^2$ and makes the integrand narrower, which also needs more nodes.

Two measurements support this. At $d = 500$ and 30% missing, doubling the inner nodes to $n_H = 128$ reduces the error from $1.2 \cdot 10^{-3}$ to $1.5 \cdot 10^{-5}$, so there the inner rule is the limit. At $d = 5000$, neither $n_H = 256$ nor $M = 19$ outer nodes bring the error below about $7 \cdot 10^{-3}$: the scale of z exceeds what both rules cover.

Holding the norm fixed. With the norm held fixed (Figure 2.2), the error of PCPG is at the precision of the single precision floating point numbers used in the computation,

for every d. The scale of z thus explains the whole loss of accuracy at high dimensions. Since the unit norm is smaller than the norm of about 2.2 that the default classifier has at low dimensions, PCPG is also more accurate there than with the default classifier at small d. Rescaling is a controlled experiment, not a way to fix the classifier, since it keeps the overfitted direction of w. In practice, stronger regularisation keeps the norm small and also improves the classifier: with C chosen such that $\|w\| = 2.2$ at $d = 5000$, the test accuracy rises from 80.5% to 83.6% and the test log loss falls from 1.22 to 0.40.

Runtime. At the same budget, PCPG evaluates the characteristic function at 1,216 frequencies per test point and MC draws 1,216 samples. PCPG is faster in our implementation mostly because MC draws all d features of every sample, including the observed ones.

Summary. The accuracy of PCPG with a fixed number of nodes depends on the scale of $z = w^T x$, that is on m_k and v_k relative to the quadrature rules, and not on the number of features as such. The argument of Section 3.2 holds for a regularised classifier. An overfitted classifier makes z large, and then more nodes are needed than a rule chosen for small z provides. In practice, the classifier should be regularised, which also improves it. Alternatively, the number of inner nodes can be chosen per test point from the largest $|m_k|$ and v_k.

3.4. Training with Missing Features

This section explains the training results of Section 2.2.

3.4.1. Monte Carlo Inside the Loss

The obvious alternative to PCPG is to estimate the expectation in the loss by sampling,

$$\hat{p} = \frac{1}{S} \sum_{s=1}^{S} \sigma\left(w^T x^{(s)}\right), \qquad x^{m,(s)} \sim p(x^m \mid x^o),$$

with new samples in every step. The samples do not depend on w, so the gradient is simply the average of the gradients at the samples. The estimate $\hat{p}$ itself is unbiased, but the loss is not: since log is concave, Jensen's inequality gives

$$\mathbb{E}\left[-\log \hat{p}\right] \geq -\log \mathbb{E}[\hat{p}] = -\log p,$$

so the Monte Carlo loss is too large on average. A second-order expansion of log around p gives the size of this bias,

$$\mathbb{E}\left[-\log \hat{p}\right] + \log p \approx \frac{\mathrm{Var}[\hat{p}]}{2p^2} = \frac{\mathrm{Var}[\sigma(z)]}{2Sp^2},$$

where $\mathrm{Var}[\sigma(z)]$ is the variance of the prediction over the missing features. The bias decreases like $1/S$, but it does not average out over the steps of the optimisation. It acts as an additional penalty on how much the prediction depends on the missing features, and it is minimised by making w smaller. The Monte Carlo objective therefore has its optimum at a smaller w than the marginal likelihood. For $S = 1$ the loss is the average loss over a single random completion of every point, $\mathbb{E}[-\log \sigma(\pm w^T x)]$, which is the objective of training on randomly imputed data (multiple imputation). With growing S, the Monte Carlo objective moves from this objective to the marginal likelihood.

3.4.2. Interpretation of the Results

The marginal likelihood is reached. The baselines minimise a different objective, so their gap does not shrink with more training: it is the price of ignoring the uncertainty of the missing features. PCPG follows Exact closely, i.e. its quadrature error does not affect the training. MC lies slightly above it: with $S = 320$ samples its bias $\mathrm{Var}[\sigma(z)]/(2Sp^2)$ is small, but visible. At both ends of the range of d the gaps are larger for all three methods including Exact, so the cause is the optimiser and not the expectation: Adam with a fixed learning rate does not settle exactly at the optimum. At 5 features it has to reach large weights ($\|w^*\| \approx 3$), and Exact stops $7.8 \cdot 10^{-4}$ above the optimum at 30% missing, more than the complete-data classifier ($1.1 \cdot 10^{-4}$). At 100 features there are many weights, and all three stop at about $6 \cdot 10^{-4}$ for 30% and 50% missing. PCPG falls behind Exact with 5 features and 30% missing because only a single feature is hidden there, so $z \mid x^o$ is concentrated and the characteristic function hardly decays, which is the hardest case for the fixed nodes of PCPG.

Accuracy. On incomplete test points, the complete-data classifier is optimal for complete points, not for the expected prediction it is used with, while the marginal likelihood trains exactly for that prediction. This is why the marginal likelihood models are more accurate at 80% missing. On complete test points, training on incomplete data necessarily loses some of the classifier, and the loss grows with the missing rate.

Adam against an exact solver. At 5 features, Adam does not reach the optimum of the logistic regression. The objective gaps of the Adam and scikit-learn fits then differ

in both directions, since the gap is measured on the marginal likelihood and not on the objective the two fits minimise. This is the same limitation of the optimiser as for Exact, and it does not change the comparison between the models.

Summary. PCPG makes the marginal likelihood usable as a training loss: the training with PCPG reaches the same classifier as training with the exact expectation, without sampling and without the systematic bias of Monte Carlo. Training on the marginal likelihood makes the classifier better suited to incomplete test points than training on complete data, with gains of up to two percentage points of accuracy at 80% missing features. On complete test points, it loses less than mean imputation at high missing rates and moderate dimensions. In terms of accuracy, the differences between the methods are small on this data, while in terms of the objective they are clear: only the marginal likelihood models reach its optimum.

3.5. Special Case: Closed Form for GMMs

For a fixed γ, the integral over u together with the expectation over x^m only undoes the HS step: they compute

$$g(\gamma) := \mathbb{E}_{p(x^m \mid x^o)}\left[\exp\left(\tfrac{1}{2}z - \tfrac{1}{2}\gamma z^2\right)\right], \qquad p(y \mid x^o) = \tfrac{1}{2}\,\mathbb{E}_{\gamma \sim \mathrm{PG}(1,0)}[g(\gamma)]\,.$$

HS is needed because, for a general density, $g(\gamma)$ cannot be computed directly. For a GMM with diagonal covariances it can. Within component k, $z = w_o^T x^o + w_m^T x^m$ is a constant plus a weighted sum of independent Gaussians, so

$$z \mid k \sim \mathcal{N}(m_k, v_k), \qquad m_k = w_o^T x^o + w_m^T \mu_k^m, \qquad v_k = \sum_{d \in \mathrm{mis}} w_d^2\, \sigma_{k,d}^2.$$

The expectation of $\exp(\frac{1}{2}z - \frac{1}{2}\gamma z^2)$ under a Gaussian is a Gaussian integral (both exponents are quadratic in z), and completing the square in z gives

$$g(\gamma) = \sum_{k=1}^{K} \tilde{\pi}_k(x^o)\, \frac{1}{\sqrt{1+\gamma v_k}} \exp\left(\frac{m_k + \frac{1}{4}v_k - \gamma m_k^2}{2\,(1+\gamma v_k)}\right).$$

For $v_k = 0$ (nothing missing) this reduces to the PG identity itself, as it should. Only the outer integral over γ remains, and it can be handled with MC or the Gauss–PG rule as before. The integrand is real, positive and bounded, so the small-γ problems discussed above do not occur, and each value of γ costs K operations instead of $K \cdot n_H$.

This does not carry over to general probabilistic circuits. A PC with Gaussian leaves is also a mixture of Gaussians, but with exponentially many components (induced trees) that cannot be enumerated. The circuit can only compute expectations of functions that factorise over the features, and $g(\gamma)$ does not, because of the γz^2 term. After the HS step the exponent is linear in z and factorises, which is exactly why PCPG needs the characteristic function. For GMMs the closed form is therefore mainly useful as an exact reference: it isolates the error introduced by the inner quadrature of PCPG.

3.6. Related Work

Here, we just focus on works that also try to calculate

$$\mathbb{E}_{P(x^m|x^o)}\left[\sigma(w^T x)\right]$$

but use a different approach.

3.6.1. Naive Conformant Learning (NaCL)

Khosravi et al. [4] first point out that this expectation is hard to compute in general: even with a naive Bayes feature distribution, it is NP-hard. They also note that the popular shortcut of mean imputation only matches the expectation when the classifier is linear *and* the features are independent — for the sigmoid it is always biased.

Their fix is to pick a feature distribution on which the expectation *is* cheap, namely a naive Bayes model over binary features. The trick is that naive Bayes and logistic regression are dual to each other: every naive Bayes induces a logistic regression, and many naive Bayes models induce the same one. They call a naive Bayes *conformant* with the given LR if both give the same class probabilities on fully observed inputs, and learn such a distribution via geometric programming ("naive conformant learning"). Once this naive Bayes is fitted, the expected prediction is just a marginal inference query and runs in linear time.

Difference to us: they keep the feature distribution simple (naive Bayes on binary inputs) so the expectation is exact and cheap; we keep the feature distribution rich (a GMM or PC on continuous inputs) and instead use a stochastic trick to handle the sigmoid. They also have to refit the feature distribution whenever the classifier changes, while ours is trained once and reused.

3.6.2. Joint Modelling via SAEM

Jiang, Josse, and Lavielle [5] assume a joint Gaussian model on the covariates, $x \sim \mathcal{N}(\mu, \Sigma)$, together with logistic regression on top. Their main contribution is on the training side: they fit (μ, Σ, β) jointly from training data that may itself contain missing values, using a stochastic-approximation EM algorithm with Metropolis–Hastings updates.

For prediction at test time, they simply do plain Monte Carlo: sample $x^{m,(s)} \sim p(x^m \mid x^o)$ from the fitted Gaussian and average,

$$p(y \mid x^o) \approx \frac{1}{S} \sum_{s=1}^{S} p\Big(y \mid x^o, x^{m,(s)}\Big)$$

This is exactly the MC baseline we compare against, just with a single Gaussian instead of a mixture as the density.

Difference to us: they sample the missing features and average the sigmoid; we never sample the missing features, but instead rewrite the sigmoid so that only the characteristic function of $p(x^m \mid x^o)$ is needed. Their feature distribution is also limited to a single Gaussian, while ours can capture multimodal shapes.

Appendix

Appendix A.

Intuitive Derivations

A.1. Bells as Waves

The Hubbard-Stratonovich identity writes a bell as an expectation of $\exp(-iu(z-c))$ over $u \sim \mathcal{N}(0,\gamma)$, with $c = \frac{1}{2\gamma}$. To see the waves, split each term of the expectation with Euler's formula $e^{-ix} = \cos x - i \sin x$:

$$\exp\left(-iu\left(z-c\right)\right) = \cos\Big(u(z-c)\Big) - i\sin\Big(u(z-c)\Big).$$

The sine is odd in u, and since u and $-u$ are equally likely under $\mathcal{N}(0,\gamma)$, it cancels and only the cosine remains:

$$\exp\left(\frac{1}{2}z - \frac{1}{2}\gamma z^2\right) = \exp\left(\frac{1}{8\gamma}\right)\mathbb{E}_{u\sim\mathcal{N}(0,\gamma)}\left[\cos\Big(u(z-c)\Big)\right].$$

A.2. The Characteristic Function as an Area

The characteristic function is used to compute $\mathbb{E}_{p(x^m|x^o)}\left[\exp\left(-iu\left(z-c\right)\right)\right]$. The integrand depends on x^m only through $z = w^T x$, so we can take the expectation over the distribution $p(z)$ of z instead. The real part of $\exp(-iu(z-c))$ is the wave $\cos(u(z-c))$ from Appendix A.1:

$$\mathrm{Re}\left[\mathbb{E}_{p(x^m|x^o)}\left[\exp\left(-iu\left(z-c\right)\right)\right]\right] = \mathbb{E}_{p(z)}\left[\cos\Big(u(z-c)\Big)\right] = \int \cos\Big(u(z-c)\Big)\,p(z)\,\mathrm{d}z.$$

This is the area under the wave times $p(z)$, where parts below zero count negatively. So for every frequency u, the characteristic function gives the signed area under the wave times $p(z)$.

A.3. Other Rules for the PG Integral

Besides Gauss–PG (Section 1.2.2), we consider three families of rules for the outer integral over $\gamma \sim \mathrm{PG}(1,0)$: trapezoid, Gauss–Legendre and Gauss quadrature, each with the nodes spaced in γ and in $\log\gamma$, and rules whose weights are fitted to the sigmoid. All rules except Gauss–PG need the density p of $\mathrm{PG}(1,0)$ pointwise. It has no closed form, but two alternating series, one converging fast for small and one for large γ, evaluate it to machine precision. All weights are normalised to sum to one.

A.3.1. Spaced in γ and in $\log\gamma$

Trapezoid. The composite trapezoid rule on $[0,T]$ with n equally spaced nodes γ_j and spacing h has the weights $w_j \propto c_j\, p(\gamma_j)\, h$, with $c_j = \frac{1}{2}$ at both ends and 1 otherwise. We use $T = 5$, above which $\mathrm{PG}(1,0)$ has a mass of only about $2 \cdot 10^{-11}$. With equal spacing, only few nodes fall where the mass is, around $\gamma \approx 0.1$ to 0.5.

Log-trapezoid. Substituting $\gamma = e^s$ turns the expectation into

$$\mathbb{E}_{\gamma\sim\mathrm{PG}(1,0)}[f(\gamma)] = \int_{-\infty}^{\infty} f(e^s)\, p(e^s)\, e^s \,\mathrm{d}s.$$

Applying the trapezoid rule with n equally spaced nodes s_j and spacing h gives nodes $\gamma_j = e^{s_j}$ and weights $w_j \propto p(\gamma_j)\,\gamma_j\, h$. Since p decays like $e^{-1/(8\gamma)}$ towards 0 and like $e^{-\pi^2\gamma/2}$ towards infinity, the range $\gamma \in [10^{-3}, 10]$ covers all of the mass. Equal spacing in $\log\gamma$ places many nodes at small γ, exactly where $e^{-\gamma z^2/2}$ lives for large $|z|$, and for smooth integrands that vanish at both ends the trapezoid rule converges exponentially in n.

For the sigmoid alone, this makes the log-trapezoid rule more accurate than Gauss–PG with the same number of nodes. Inside PCPG, however, every node comes with the factor $e^{1/(8\gamma_j)}$ of f, which multiplies the error of the inner expectation. At the smallest node $\gamma = 10^{-3}$ this factor is about 10^{54}, which overflows in floating point and makes the estimate useless. The smallest Gauss–PG node for $n = 19$ is $\gamma \approx 0.046$, where the factor is below 16.

Gauss–Legendre. The standard Gauss–Legendre nodes $x_j \in [-1,1]$ with weights v_j are mapped to $\gamma_j = T(x_j+1)/2$ on the same interval $[0,T]$, and the density goes into the weights, $w_j \propto v_j\, \frac{T}{2}\, p(\gamma_j)$. The rule is exact for polynomials times the density, which is not what we integrate, so it does not reach the convergence rate of a Gaussian rule for $\mathbb{E}[f(\gamma)]$.

Log-Gauss–Legendre. The same in $s = \log \gamma$, on the same range as the log-trapezoid rule: the Legendre nodes are mapped to $[\log 10^{-3}, \log 10]$, and the weights get the factor $p(e^s)\, e^s$ as there.

Log-Gauss–PG. The Gaussian quadrature rule in $s = \log \gamma$, with the distribution of s as the weight function, so it is exact for polynomials in s up to degree $2n - 1$. The moments of s have no closed form, so the Jacobi matrix is not built from moments but by the Lanczos (Stieltjes) procedure on a fine discretisation of the distribution of s, with 4000 trapezoid points on $[\log 10^{-4}, \log 20]$. Lanczos builds the orthonormal polynomials point by point, with reorthogonalisation so that it stays stable, and their recurrence coefficients form the Jacobi matrix. The nodes and weights then follow from its eigendecomposition as in the Golub–Welsch algorithm.

A.3.2. Fitting the Nodes to the Sigmoid

With the sign flip (Section 1.2.2), the rule is only evaluated for z around and below 0, and the error that matters is $\frac{1}{2}e^{z/2}\varepsilon(z^2/2)$ on that side. None of the rules above optimise this quantity, since they all discretise the $\mathrm{PG}(1, 0)$ distribution.

Since the rule enters the estimator linearly, the estimator computes exactly $\mathbb{E}[\hat{\sigma}(z)]$, and its error is the error of $\hat{\sigma}$ averaged over the distribution of z. Nothing in the rest of the method requires the nodes to represent $\mathrm{PG}(1, 0)$: HS and the characteristic function only need $\gamma_j > 0$, and the weights only need to be nonnegative. We can therefore choose the nodes and weights to approximate the sigmoid itself. On a grid of 400 log-spaced candidate nodes $\gamma_j \in [0.02, 8]$ and 1500 fitting points $z_r \in [-30, 5]$, we minimise a norm of the error curve

$$e_r = \tfrac{1}{2}\, e^{z_r/2} \sum_j w_j\, e^{-\gamma_j z_r^2/2} - \sigma(z_r)$$

over nonnegative weights that sum to one. We use three norms:

- **L2**, the sum of squared errors, solved by nonnegative least squares with the sum-to-one constraint as an extra row,
- **L1**, the sum of absolute errors, i.e. the area under the error curve,
- **max**, the largest absolute error, which makes the error curve equally high everywhere.

L1 and max are linear programs. All three keep only a few of the candidates, so the number of nodes is not chosen directly. The sum-to-one constraint makes the estimate exactly $\frac{1}{2}$ at $z = 0$. The margin up to $z = 5$ covers the part of the distribution of z that remains on the positive side after the flip, and the grid starts at $\gamma = 0.02$, so that $e^{1/(8\gamma)}$ in the inner integral stays below about 500.

This changes the view of the outer integral: the method approximates the sigmoid by functions of the form $\frac{1}{2}e^{z/2}e^{-\gamma z^2/2}$, whose expectations are tractable, and the PG identity is one particular, exact but infinite, choice of how to combine them. A finite rule only has to combine them well, not to represent the PG distribution. The only variants this does not apply to are the ones that sample γ from $\mathrm{PG}(1,0)$, since a fitted rule cannot be sampled from.

A.4. Variants of PCPG

Each of the two expectations of PCPG can be computed by quadrature or by Monte Carlo, which gives the four combinations in Table A.1. The variants with Monte Carlo use Algorithm 3 with the sampled nodes and the equal weights $\frac{1}{M}$ and $\frac{1}{N}$ in place of the quadrature nodes and weights, and without the factor $\frac{1}{\sqrt{\pi}}$ of Gauss–Hermite.

Variant	**Outer** (γ_j, w_j)	**Inner** (u_{ij}, ω_{ij})
PCPG-MC	$\gamma_j \sim \mathrm{PG}(1,0)$, $w_j = \frac{1}{M}$	$u_{ij} \sim \mathcal{N}(0, \gamma_j)$, $\omega_{ij} = \frac{1}{N}$
PCPG-GH	$\gamma_j \sim \mathrm{PG}(1,0)$, $w_j = \frac{1}{M}$	Gauss–Hermite: $u_{ij} = \sqrt{2\gamma_j}\, t_i$, $\omega_{ij} = \frac{\omega_i}{\sqrt{\pi}}$
PCPG-GPG	Gauss–PG rule	$u_{ij} \sim \mathcal{N}(0, \gamma_j)$, $\omega_{ij} = \frac{1}{N}$
PCPG	Gauss–PG rule	Gauss–Hermite: $u_{ij} = \sqrt{2\gamma_j}\, t_i$, $\omega_{ij} = \frac{\omega_i}{\sqrt{\pi}}$

Table A.1.: The four combinations of Monte Carlo and quadrature for the two expectations of PCPG. Hermite nodes (t_i, ω_i) as in Section 1.2.2, Gauss–PG nodes (γ_j, w_j) as in Section 1.2.2.

PCPG-GPG samples the inner expectation, whose variance is very high, so it performs much worse than PCPG in practice.

A.5. Why Large Positive z Is Hard

In (1.5), any error ε a rule makes in approximating $\mathcal{L}(z^2/2)$ enters the estimate of $\sigma(z)$ multiplied by $\frac{1}{2}e^{z/2}$. The nodes only see z^2, so this error is the same for z and $-z$, but its effect is not:

$$\left|\hat{\sigma}(z) - \sigma(z)\right| = \tfrac{1}{2}\, e^{z/2}\, \varepsilon(z^2/2).$$

For $z < 0$ the factor shrinks the error, for $z > 0$ it amplifies it exponentially. Staying at $\sigma(z) \approx 1$ for large positive z requires the sum of exponentials to reproduce the tail $\mathcal{L}(z^2/2) \approx 2e^{-z/2}$ almost exactly, which only rules with nodes very close to $\gamma = 0$ can do. Rules with few or no nodes there (trapezoid, Gauss–Legendre, Gauss–PG) have a

tail that is too small, so $\hat{\sigma}(z)$ falls back to 0 and the error approaches 1. A rule that puts more weight at tiny γ than $\mathrm{PG}(1, 0)$ really has there does the opposite: its tail is too heavy, and $\hat{\sigma}(z)$ blows up like $e^{z/2}$. Only the rules spaced in $\log \gamma$ stay accurate without further help, and only with enough nodes.

Bibliography

[1] Nicholas G Polson, James G Scott, and Jesse Windle. "Bayesian inference for logistic models using Pólya–Gamma latent variables." In: *Journal of the American statistical Association* 108.504 (2013), pp. 1339–1349 (cit. on p. 1).

[2] R. L. Stratonovich. "On a method of calculating quantum distribution functions." In: *Soviet Physics Doklady* 2 (1957), pp. 416–419 (cit. on p. 2).

[3] J. Hubbard. "Calculation of partition functions." In: *Physical Review Letters* 3.2 (1959), pp. 77–78. DOI: 10.1103/PhysRevLett.3.77 (cit. on p. 2).

[4] Pasha Khosravi et al. "What to expect of classifiers? Reasoning about logistic regression with missing features." In: *Proceedings of the 28th International Joint Conference on Artificial Intelligence (IJCAI)*. 2019. URL: https://arxiv.org/abs/1903.01620 (cit. on p. 38).

[5] Wei Jiang, Julie Josse, and Marc Lavielle. "Logistic regression with missing covariates—Parameter estimation, model selection and prediction within a joint-modeling framework." In: *Computational Statistics & Data Analysis* 145 (2020), p. 106907. URL: https://arxiv.org/abs/1805.04602 (cit. on p. 39).